ECS Pre-Trip Travel Request — Technical Reference

Audience: RFP respondents / build team & business analysts **Basis:** working validation prototype

Generated: 2026-10-22

REQUEST FOR PROPOSAL — TECHNICAL APPENDIX

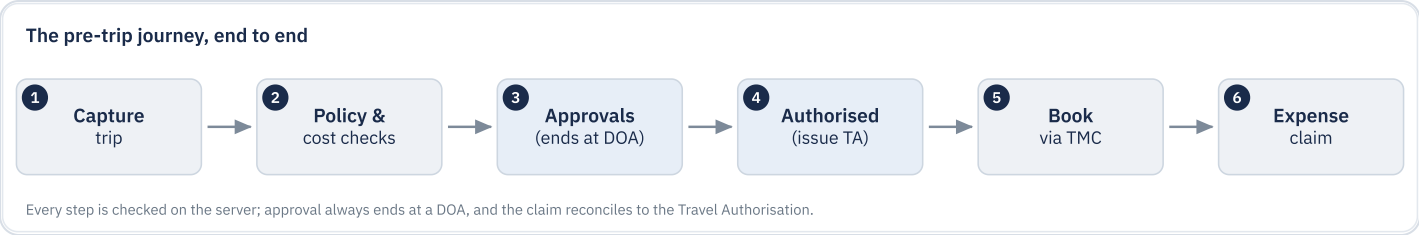
Document	Technical Reference for RFP respondents / implementation build team
Module	ECS Pre-Trip Travel Request (extension of the NTU Employee Claim System)
Version	1.0 — draft for review
Basis	Working validation prototype — Next.js 16 · React 19 · TypeScript · Prisma 6 · SQLite
Subject	Pre-trip travel request, approval, TMC hand-off and expense-claim seeding built on the existing ECS/ECS-Travel platform

> This document describes the behaviour of a requirements-validation prototype. It is complete enough to be read as an **executable specification** — every rule, algorithm and field below is implemented in the prototype and demonstrable. Statements marked **Derived requirement (TR-PT-nn)** are proposals derived from that behaviour and remain subject to confirmation by the University. The prototype is a reference for *what* must be built, not a mandate for *how*. Where this document and the demonstration differ, this document records the intended behaviour; Chapter 14 records the places where the prototype deliberately simplifies.

Executive summary

What it is. The ECS Pre-Trip Travel Request module extends NTU's Employee Claim System to govern a trip *before* it happens. It captures the request, works out the traveller's entitlement and cost, applies travel policy, routes it to the right approvers, issues a dated **Travel Authorisation (TA)**, hands a governed booking instruction to the travel agency (TMC), and pre-fills the post-trip expense claim so the claim reconciles to what was approved.

The problem it solves. Today, travel is often approved over email and booked before it is authorised, so policy is discovered at claim time and spend is not controlled up front. This module puts **policy and financial authority before the booking**: caps, cabin entitlement, duplicate trips and exceptions are checked and routed to the correct Delegation-of-Authority (DOA) holder, and only then is the trip booked — against a single authorisation the agency and the claim both reconcile to.



What it does (at a glance).

- Individual, group and guest requests
- Round-trip / one-way / multi-city itineraries
- Automatic travel-class entitlement and cost (auto-derived per-city hotel and overseas allowance, with a long-trip taper)
- A configurable policy engine (pass / warning / exception / hard stop)
- Configurable approval routing that always ends at a DOA and never lets anyone approve their own trip
- Flexible payment settlement (central account vs individual claim)
- A governed TMC hand-off with group fan-out and multi-PNR handling
- Booking retrieval keyed by the authorisation
- Expense-claim seeding with automatic DOA sign-off when the claim matches

Guiding principles. Every rule runs on the server and cannot be bypassed from the screen; separation of duties is enforced (an approver cannot raise a request they might approve); policy, thresholds and routing are **configuration, not code**; one live authorisation per trip; and a full audit trail with immutable approved versions.

Status. The behaviour described here is implemented and demonstrable in a working validation prototype. This document is the build contract; Chapter 14 lists what a production build must add (live integrations, enterprise identity, exact-decimal money, an automated test suite) and Appendix F estimates the OutSystems build size.

How to read this document

READER	WHAT THEY NEED	START WITH
Solution architect	Scope, boundary, layering, integration, production gaps	Ch 1–3, 9, 14
Data / integration engineer	Entity model, field dictionary, versioning, interface payloads	Ch 4, 5, 9, 12
Application developer	Runtime algorithms, service & action contracts, error model, state machines, screens	Ch 6, 7, 10, 11
Security / controls reviewer	Separation of duties, access derivation, scope, audit, enforcement gaps	Ch 8, 14
Test lead / evaluator	Acceptance criteria, test inventory, open decisions, demo script	Ch 13, 14, App. D
Procurement panel	Scope, derived requirements, gap register, NFRs	Ch 1, 2, 14

Conventions. Keywords **MUST / SHOULD / MAY** are used in the RFC sense (MUST = mandatory, SHOULD = strongly recommended, MAY = optional). A *Derived requirement* (TR-PT-nn) states a requirement implied by prototype behaviour. A *Design decision* (DD-nn) is a binding decision documented in the companion design Build Spec (App. D). A *Decision point* (O-nn) is an open policy item for the University to confirm. Code identifiers (field names, functions) are shown in `monospace` — a business reader can treat them as labels and skip them. Money is SGD. Dates are the official travel dates unless stated.

Reading this as a business analyst. This is a technical document, but it is written to be followed without coding knowledge. Each chapter and each rule opens with a plain-English explanation of *what it does and why*; the tables, code snippets and `monospace` names that follow are detail for the build team. If you read only the opening sentences of each section, the **tables** (which are plain-language), Chapter 2 (overview), Chapter 6's lead-ins (what the system decides and how), and Chapter 14 (what differs from a production build), you will have the full business picture. Technical terms (for example *idempotent*, *effective-dated*, *roll-up*) are defined in plain words in **Appendix A — Glossary**.

Chapter 1 — Introduction

1.1 Purpose

This is the technical appendix to the RFP for the **ECS Pre-Trip Travel Request** module. It describes, at developer level: the data the module owns and how it links; the configuration that expresses NTU travel policy; the algorithms that derive entitlement, cost, policy outcomes and the approval route; the controls that govern submission, approval and change; and the boundaries with neighbouring systems (ECS core, HR/Workday, DOA, SAP Finance, the TMC/OBT, and the downstream TE claim).

1.2 Scope & system boundary

In scope (built by this module): the pre-trip lifecycle from *draft request* to *approved Travel Authorisation*, the governed *TMC hand-off*, *booking reconciliation*, and the *seeding of the downstream TE claim*. The module owns the transactional store in §4.2 and the runtime Module Settings (§5.1), the Integration-Contract overrides (§9) and the Provost list. **Out of scope (reused read-only from ECS):** HR identity & appointments, reporting lines, DOA lines & amount bands, charging accounts (CC/WBS), policy caps/ODA/class register, FX rates, the location/airport master, TMC providers, GST/GL codes. The module **reads** these through the single service boundary in §2 (Ch 10) — it never maintains them. **Downstream:** the TE claim is created *from* the Travel Request (Ch 6.13, Ch 10) but the claim's own post-trip lifecycle beyond create/pre-populate/auto-grant is the ECS TE module's.

1.3 What is real and what is simulated

AREA	REAL IN THE PROTOTYPE	SIMULATED / TO BE WIRED IN PRODUCTION
Lifecycle, routing, policy, cost, settlement, duplicate, class entitlement	Fully implemented, server-enforced	—
Reference masters (employees, DOA lines, caps, ODA, bands, FX, locations)	Seed data read through the §2 services	Replace with live ECS/HR/DOA/Finance reads
TMC hand-off & inbound	Canonical payload built; inbound **simulated** by an adapter	Replace the adapter with the real TMC/OBT interface
SAP airfare posting, visa-letter batch, notifications	Modelled as actions/audit	Wire to SAP, immigration, notification channels
File storage (attachments)	Bytes stored in-row	Replace with a document store
Authentication	Persona cookie switcher	Replace with ECS SSO/identity (§8.2)

1.4 Provenance

The requirements are derived from the ECS Pre-Trip Build Pack and the CFO review decisions (DD-01…DD-30, App. D), validated by the running prototype. Companion artifacts: the **design Build Spec** (decisions + screen narratives) and the **Feature Walkthrough** (live screens).

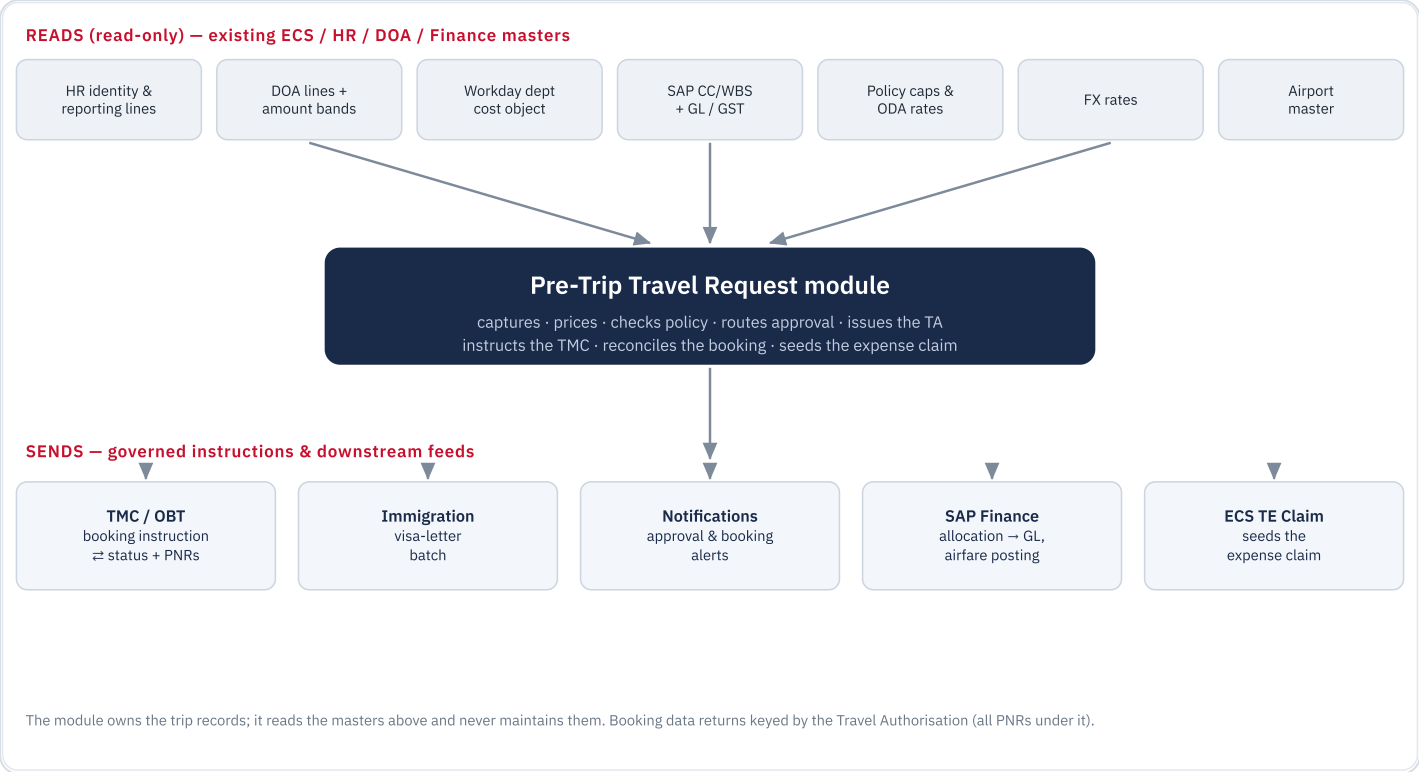
TR-PT-01 The solution MUST implement the full pre-trip lifecycle (draft → approval → Travel Authorisation → TMC hand-off → booking reconciliation → claim seeding) with policy and routing enforced **server-side**, never trusting the client. **TR-PT-02** The solution MUST read all reference masters (identity, DOA, charging, policy rates, FX, locations, bands) through a single integration boundary and MUST NOT maintain copies of them.

Chapter 2 — Solution overview

2.1 What the module does

It closes the gap between an employee's **intent to travel** and a **booked, policy-compliant, financially authorised trip**: capture the trip; derive entitlement (travel class) and cost (airfare, cap-checked accommodation, tiered ODA); evaluate policy (caps, class, duplicates, risk); route for approval (configurable RO / Additional / Funding / Cross-BA / Exception / Provost / **DOA**); issue a dated, amount-bound **Travel Authorisation (TA)**; transmit a governed booking instruction to the TMC; reconcile the booking (PNRs, fares, deviation); and seed the post-trip **TE claim** so it reconciles to what was approved.

2.2 System context



2.3 Actors / personas

`ROLE` = `TRAVELLER, TRAVEL_REQUESTOR, RO, DOA, RESEARCH_DOA, EXCEPTION_APPROVER, TRAVEL_ADMIN, SYSTEM_ADMIN, FINANCE_VIEWER` (plus the guest **Provost** authority, DD-25). The prototype's persona switcher (`PERSONA_ROLES`, grouped into REQUEST / CLAIM / ADMIN stages) selects who you act as; in production this is ECS SSO (§8.2). **Creation remit** and **approver entitlement** are role-derived and enforced server-side (§8).

2.4 Core concepts

CONCEPT	MEANING	REFERENCE
Travel Request (TR)	The governed transactional record; `TR-YYYY-NNNNNN`.	§4, Ch 6
Travel Authorisation (TA)	Dated, amount-bound authority issued on final approval; `TA-YYYY-NNNNNN`.	§6.9
Estimate line	Airfare / Accommodation / ODA / Conference / Other; ODA + hotel are **auto-derived per city**.	§6.1
Approval route	Ordered steps computed from context + Module Settings; always ends at a DOA.	§6.6
Policy check	PASS / WARNING / EXCEPTION / HARD_STOP outcome per rule.	§6.4
Settlement channel	Who fronts the money (CTA/central vs employee) → reimbursable vs reference-only (DD-19).	§6.10
Booking / PNR	One TR → **N PNRs** (multi-city split); retrieval is **TA-keyed** (DD-17).	§6.12, §9.4

2.5 Capability map

The module's capabilities group as follows:

- Request capture (individual / group / guest)
- Itinerary & trip types (round / one-way / multi-city, personal segments)
- Entitlement derivation (travel class)
- Cost estimation (auto per-city hotel + tiered ODA, airfare, FX, sponsorship)
- Policy engine (pass / warning / exception / hard stop)
- Configurable approval routing
- Charging & allocation
- Authorisation & booking lifecycle
- Governed TMC hand-off + group fan-out + multi-PNR
- Claim seeding + DOA auto-grant
- Amendments, versioning & audit
- Runtime configuration

2.6 Guardrails & design principles

- **Server-authoritative.** Every mutation re-derives and re-checks; the UI is a convenience, not a gate (§8).
- **Separation of duties.** No one approves a request they travel on or raised; approver-only personas cannot originate a request (§8.6, DD-28).
- **Configuration over code.** Routing, thresholds, caps basis, tolerances and the integration contract are runtime settings (§5).

- **One live authorisation per trip.** A true duplicate is blocked; surfaced upfront (DD-14, §6.11).
- **Policy before spend.** Caps, class and exceptions are evaluated and routed before booking, not discovered at claim.
- **One TA is the single source of truth** the TMC and the claim both reconcile against.

TR-PT-03 Approval authority (RO/DOA/Exception) MUST NOT be exercisable by a Travel or System Administrator by virtue of their admin role; an administrator may approve only when they independently hold the role **and** are the assigned approver.

Chapter 3 — Architecture reference

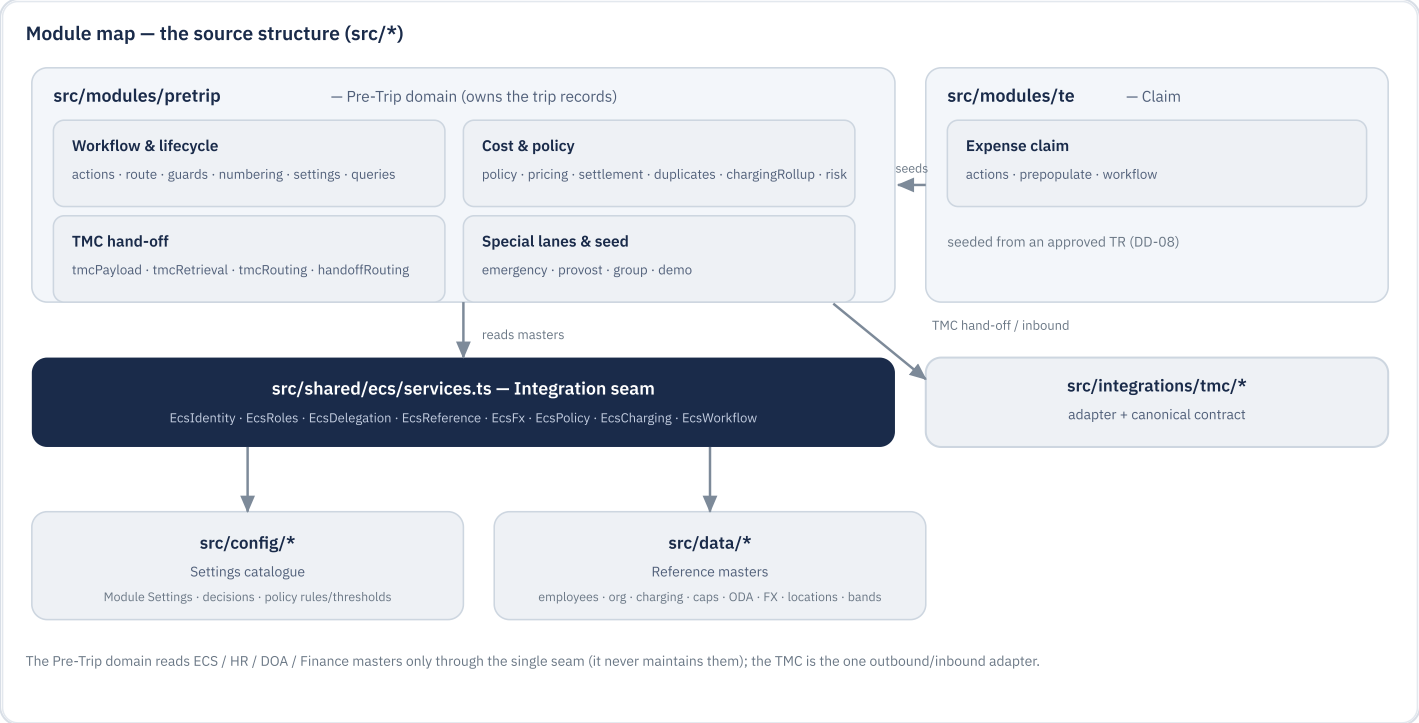
3.1 Technology stack (prototype)

Next.js 16 (App Router, Turbopack), React 19 **Server Components + Server Actions**, TypeScript, Prisma 6 ORM, SQLite. Tailwind v4 styling. No REST API layer — **all mutations are server actions** (`"use server"`); all reads are server-component data loaders. Production mapping: Screens/Blocks ← pages; Server Actions ← server actions; Entities ← Prisma models; Integration (REST / Service Actions) ← the §2 ECS services.

3.2 Layered architecture

LAYER	RESPONSIBILITY
Presentation	Wizard (Traveller→Trip→Estimates→Charging→Policy→Submit), approver/DOA queue, TE claim, admin/config, finance & reporting.
Domain logic	Route engine, class entitlement, cost/ODA/accommodation, policy evaluation, validation, charging roll-up, settlement, duplicate detection, claim workflow & auto-grant.
Integration seam	`EcsIdentity` · `EcsRoles` · `EcsDelegation` · `EcsReference` · `EcsFx` · `EcsPolicy` · `EcsTravelClassRegister` · `EcsCharging` · `EcsWorkflow` + TMC adapter + notification/visa hooks.
Transactional store	§4.2 entities.
Reference masters	Read-only ECS masters — never written.

3.3 Module map (source)



`src/modules/pretrip/*` — `actions.ts` (server actions), `route.ts` (routing), `policy.ts` (policy engine), `pricing.ts` (cost), `settlement.ts` (DD-19 + guest settlement), `duplicates.ts` (DD-14), `tmcPayload.ts` / `tmcRetrieval.ts` / `tmcRouting.ts` / `handoffRouting.ts` (TMC), `emergency.ts`, `risk.ts`, `provost.ts`, `group.ts`, `chargingRollup.ts`, `guards.ts` (access), `queries.ts` (loaders), `numbering.ts`, `settings.ts`, `demo.ts` (seed). `src/modules/te/*` — claim `actions.ts`, `prepopulate.ts`, `workflow.ts`. `src/shared/ecs/services.ts` — the integration seam. `src/config/*` + `src/data/*` — settings catalogue + reference masters. `src/integrations/tmc/*` — adapter + canonical contract. Full source map in App. C.

3.4 Request lifecycle & trust boundaries

Every server action: (1) resolves the acting persona (`currentPersonaId`); (2) checks the lifecycle source state (`actionAllowed`) and the acting remit (`canEditRequest` / `canActOnStep`); (3) mutates in a transaction; (4) writes an `AuditEvent`; (5) `revalidatePath` (soft refresh, preserves client state) or `redirect` (for a validation failure, with `?error=`). The trust boundary is the server action — the client never carries authority.

3.5 Error model

- **Validation failures** redirect back to the step with `?error=<message>` (user-facing, specific).
- **Hard stops** are `PolicyCheck` rows that disable Submit (not thrown).
- **Guard failures** (wrong state/remit) no-op and audit; actions return without effect.
- **Seed integrity** assertions throw loudly so bad seed data is caught at build.

3.6 Build, run & seed

`npm run dev`; `npm run seed` seeds the single `ModuleSettings` row; demo scenarios (28, §12.2) seed lazily when the request table is empty. **Migrations are additive** — `prisma db push` + a marked migration via `migrate diff … --script` + `migrate resolve --applied`; never `migrate dev`/reset against seeded data. SQLite has no native enum — status/type columns are `String`; production MAY use native enums or check constraints.

3.7 Architectural requirements for production

TR-PT-05 All approval, policy and routing logic MUST execute server-side and be re-evaluated on submit and on every approval, independent of UI state. **TR-PT-06** Money MUST be exact decimal; FX MUST be effective-dated and re-locked at submission. **TR-PT-07** Every material action MUST write an append-only audit event; approved/submitted states MUST snapshot an immutable version. **TR-PT-08** The TMC hand-off and inbound ingestion MUST be idempotent (a booking already sent/in-flight/booked is never re-sent or duplicated).

Chapter 4 — Data architecture (the information the system holds)

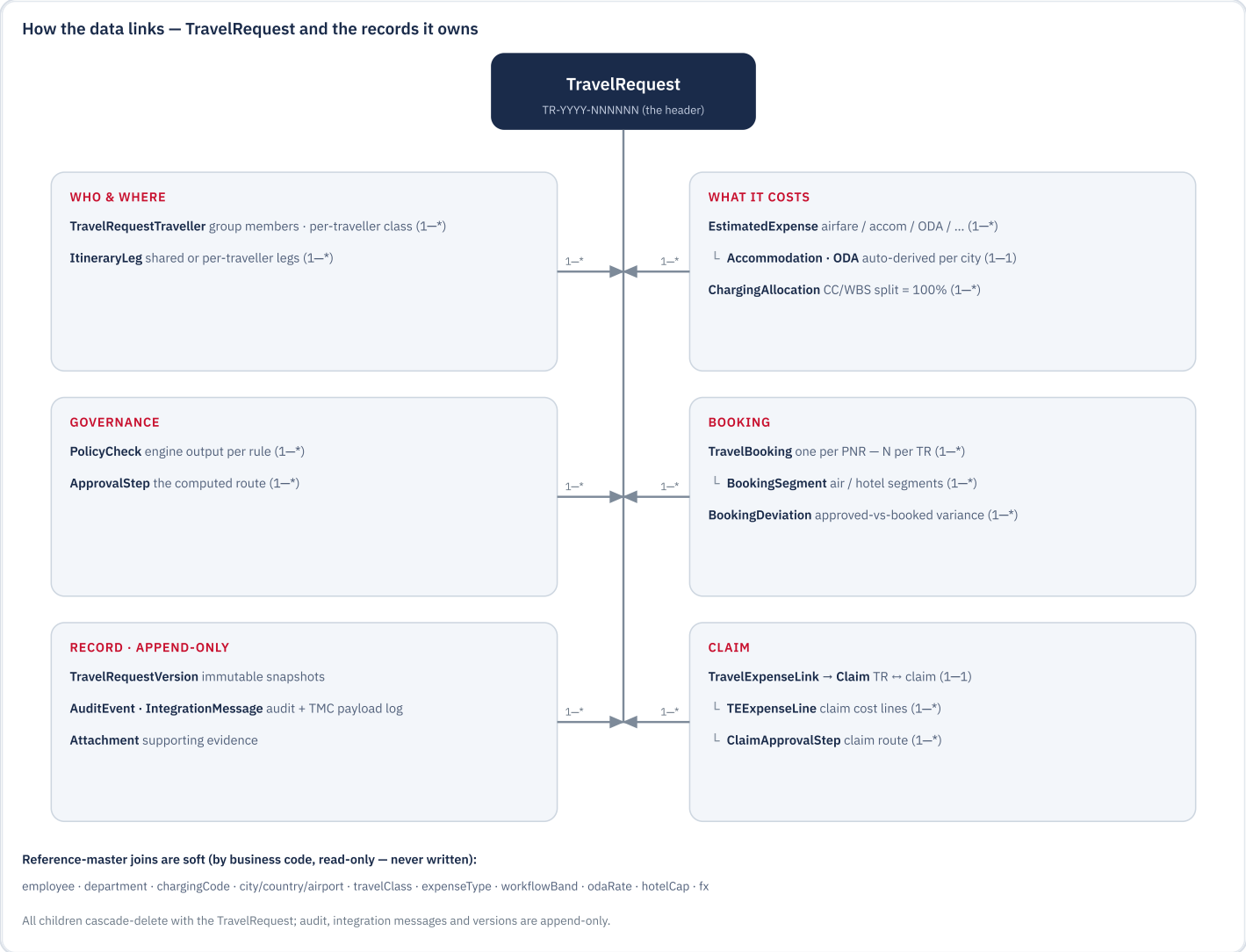
In plain terms. This chapter lists every kind of record the system stores and what each field means. The one thing to hold onto: the system **keeps the reference information** (people, approval limits, funding accounts, rates) in the existing ECS systems and only **reads** it; it *owns* the trip-specific records (the request, its legs, estimates, approvals, bookings and the claim). The field tables below are a plain data dictionary — a BA can read them directly.

4.1 Modelling conventions

CONVENTION	RULE
Row identity	Every row has a `cuid()` surrogate `id`; the TR also carries a natural number `requestNumber` (`TR-YYYY-NNNNNN`) and, once approved, `authorisationNo` (`TA-YYYY-NNNNNN`), both unique.
References	Transactional children hold a formal FK to their parent (`requestId`, `expenseId`, `bookingId`, `claimId`) with **cascade delete**. References to **reference masters** (employee ids, charging codes, city/country/airport codes, class ids, expense-type ids) are **soft** — stored as the natural code and resolved through the §2 services.
Enumerations	SQLite has no enum: status/type fields are `String` constrained in the application to the values in §4.4 / App. B.
Structured values	JSON held as text in a `String` column: `shareMap`, `allocJson`, `chargingJson`, integration `payload`, version `snapshot`. Production SHOULD use JSON/JSONB or normalised child tables.
Effective dating	Reference masters (caps, ODA, FX, class register) are effective-dated and read as-at the submission/entry date. Transactional records are versioned by `TravelRequestVersion` snapshots on submission/approval; the TR itself carries `currentVersion`.
Money & dates	Amounts are SGD (prototype `Float`; production MUST be exact decimal). Dates are `DateTime`; the **official** `startDate`/`endDate` are the governing dates for duplicates, ODA days and validation.
Append-only	`AuditEvent` and `IntegrationMessage` are write-and-read only (no update/delete route).
Correlation	Audit and integration rows carry `requestId`; a booking's PNRs are correlated to the TA (`authorisationNo`).

TR-PT-09 Submitted and approved states MUST be snapshotted as immutable versions; a superseded snapshot MUST NOT be overwritten. **TR-PT-10** Audit and integration-message records MUST be append-only with no update/delete interface.

4.2 How the data links together



Reference-master joins (soft, by code): `employee`, `department`, `chargingCode`, `city/country/airport`, `travelClass`, `expenseType`, `workflowBand`, `odaRate`, `hotelCap`, `fx`.

4.3 Entity catalogue (field-level data dictionary)

Format: **Field · Type · Constraint · Description**. `?` = nullable. All children cascade-delete from `TravelRequest` unless noted.

4.3.1 `TravelRequest` — header & lifecycle

FIELD	TYPE	CONSTRAINT	DESCRIPTION
id	String	PK, cuid	Surrogate key.
requestNumber	String	unique	`TR-YYYY-NNNNNN`.
authorisationNo	String?	unique	`TA-YYYY-NNNNNN`; set on final approval.
status	String	default `Draft`	Request state (§4.4).
bookingStatus	String	default `Not Sent`	Booking state (§4.4).
isGroup	Boolean	default false	Group request flag.
requestorId	String		Creator (may be non-traveller).
travellerId	String		Primary traveller; `GUEST` for a non-employee.
travellerType	String	default EMPLOYEE	`EMPLOYEE | GUEST`.
guestName / guestEmail / guestOrg	String?		Guest identity (no HR record; PDPA — §9).
additionalApproverId	String?		Traveller-selected extra approver.
selectedDoaId	String?		Traveller-selected DOA (not pre-selected).
provostId	String?		Selected Provost for a guest class upgrade.
isEmergency / isRetrospective	Boolean	default false	Emergency lane flags (DD-23).
emergencyJustification / emergencyAuthoriserId	String?		Emergency reason + authorising officer.
emergencyRatified / emergencyOutsideWindow	Boolean	default false	Retrospective ratification / out-of-window flag.
entityId / departmentId	String?		Owning entity / department.
isResearch	Boolean	default false	Research-charged (→ Research DOA).
purposeId / description	String?		Trip purpose + justification.
tripType	String	default ROUND_TRIP	`ROUND_TRIP | ONE_WAY | MULTI_CITY`.
destCountry / destCity	String?		Derived main destination (longest-stay leg).
startDate / endDate	DateTime?		**Official travel dates** (governing).
personalStart / personalEnd	DateTime?		Personal extension window.
personalAck	Boolean	default false	Personal-portion-at-own-cost ack.
highRiskAck / highRiskApproverAck	Boolean	default false	§4.8 acknowledgements.
visaLetterRequired	Boolean	default false	Visa-letter needed (§4.13).

FIELD	TYPE	CONSTRAINT	DESCRIPTION
visaLetterNotifiedAt	DateTime?		Set by the daily visa batch.
tmcProviderId	String?		Preferred TMC provider (null = auto).
eventStartDate / eventEndDate	DateTime?		Event/conference dates (may differ from travel).
invitationRef	String?		Invitation/acceptance reference.
travelClassId / entitledClassId / classBasis / classJustification	String?		Chosen + derived class (individual) + basis + upgrade justification.
bookingArrangement	String?	default AUTO	`AUTO | TMC-<id> | SELF_BOOKED | HOST_ARRANGED | NO_BOOKING`.
bookingMethod	String?		TMC channel `TMC_ONLINE | AGENT_ASSISTED | OFFLINE` (when TMC-arranged).
chargingMode	String	default REQUEST	`REQUEST | LINE`.
hotelSettlement	String	default INDIVIDUAL	**DD-19** `INDIVIDUAL | CTA` — hotel payment channel (§6.10).
approvalAmountSgd	Float?		Amount-basis result for DOA tiering.
bookingDeadline / authorisationExpiry	DateTime?		§13.3 booking window / TA validity.
currentVersion	Int	default 0	Version counter.
createdAt / updatedAt	DateTime	now / @updatedAt	

4.3.2 `TravelRequestTraveller` — group member

`id` PK · `requestId` FK · `employeeId` (or synthetic `G-…` for a guest member) · `travellerType` (EMPLOYEE|GUEST) · `guestName/Email/Org?` · `confirmed` + `confirmedAt?` (inclusion confirmation) · `sharePct` (default 100) · `isRequestor` · `entitledClassId?/chosenClassId?/classBasis?/classJustification?` (per-traveller class — DD-05) · `ownStartDate?/ownEndDate?/legOverrideNote?` (per-traveller sub-itinerary — §13.20).

4.3.3 `ItineraryLeg`

`id` PK · `requestId` FK · `travellerId?` (null = shared group leg; set = that traveller's sub-itinerary) · `seq` · `originCode` · `destCode` · `departDate? / arriveDate?` · `transportMode` (default AIR; AIR|RAIL|OTHER) · `durationHours?` (class derivation) · `departTime?` · `bookingRequired` (default true) · `isPersonal` (default false; excluded from ODA) · `nights` (default 0) · `travelClassId? / entitledClassId? / chosenClassId?` (per-leg class).

4.3.4 `EstimatedExpense` (+ `AccommodationEstimate`, `ODAEstimate`)

`EstimatedExpense`: `id` PK · `requestId` FK · `travellerId?` (attributed traveller; null = shared/requestor) · `category` (AIRFARE|ACCOMMODATION|ODA|CONFERENCE|OTHER) ·

`expenseTypeId` (ECS master) · `currency` (default SGD) · `foreignAmount` · `sgdAmount` · `sponsorForeign` · `sponsorSgd` · `estimateBasis?` (QUOTED|CAP|LOWER|RATE) · `expectedDate?` · `autoDerived` (default false — policy-seeded per-city line, re-derived on a dates/itinerary change) · `isShared` (default false) · `shareMap?` (JSON {employeeId:pct}) · `chargingCode?` + `allocJson?` (JSON [{code,pct,io}]=100% — LINE cross-charge) · `notes?` · airfare-specific `originCode?/destCode?/proposedClassId?/fareCeiling?/handoffStatus?`. `AccommodationEstimate` (1-1, `expenseId` unique): `city · legSeq?` (the itinerary leg this auto per-stop line maps to) `· checkIn?/checkOut? · nights · personalNights · quotedNightly · capNightly` (from ECS policy) `· budgetedNightly` (per basis) `· capVariance · conferenceHotel · exceptionOutcome?` (PASS|EXCEPTION). `ODAEstimate` (1-1, `expenseId` unique): `country · city? · legSeq? · arrive?/depart? · eligibleDays · personalDays · dailyRate · ratePct` (default 100).

4.3.5 `ChargingAllocation`

`id` PK · `requestId` FK · `companyCode? · businessArea? · chargingType` (CC|WBS) `· chargingCode` (ECS master) `· isResearch · percent · amountSgd?` (AMOUNT-mode entry → percent on save) `· internalOrder? · isMain` (the main charging account). **Must total 100%** across the request (AC06).

4.3.6 `PolicyCheck`

`id` PK · `requestId` FK · `code` · `label` · `outcome` (PASS|WARNING|EXCEPTION|HARD_STOP) · `detail?` · `travellerId?` (per-traveller check) · `resolved` (exception approved).

4.3.7 `ApprovalStep`

`id` PK · `requestId` FK · `seq` · `roleType` (RO|DOA|RESEARCH_DOA|EXCEPTION|FUNDING_OWNER|ADDITIONAL_APPROVER|PROVOST) · `approverId?` · `onBehalfOf?` (delegation) · `status` (default Pending; Pending|Approved|Rejected|SentBack|Skipped) · `decidedAt?` · `comments?`.

4.3.8 `TravelBooking` (+ `BookingSegment`, `BookingDeviation`)

`TravelBooking`: `id` PK · `requestId` FK · `travellerId?` (group fan-out) · `tmcProviderId?` · `pnr?` · `ticketNo?` · `channel?` (TMC|SELF_BOOKED) · `fare?/taxes?/fees?/hotelRate?` · `status` (default Booked). **One TR → N bookings/PNRs** (DD-17). `BookingSegment`: `id` PK · `bookingId` FK · `type` (AIR|HOTEL) · `originCode?/destCode?` · `bookedClassId?` · `departDate?/arriveDate?` · `amount?`. `BookingDeviation`: `id` PK · `requestId` FK · `field` · `approvedValue?/bookedValue?` · `material` · `action?` (Reapproval|LogOnly|UpdateBooking).

4.3.9 TE claim entities

`TravelExpenseClaim`: `id` PK · `claimNumber` unique · `requestId?` FK · `claimantId` · `status` (Draft|Submitted) · `reportType` · `trsNotBooked` · `travelStart?/End?` · `additionalApprover1?/2?` · `lessCorpCard · lessPersonal` · `chargingMode` (MAIN|CLAIM|ITEM) `· chargingMainCode? · chargingJson?` (JSON rows + lineMap).

`ClaimApprovalStep`: `id` PK · `claimId` FK · `seq` · `roleType` (VERIFIER|EXCEPTION|RO|DOA|RESEARCH_DOA) · `approverId?` · `status` · `autoGranted` (DOA auto-granted from the TR) · `decidedAt?/comments?`. `TEExpenseLine`: `id` PK · `claimId` FK · `category · expenseTypeId · treatment` (LOCKED|EDITABLE|REFERENCE|EXCLUDED) · `approvedSgd · bookedSgd?` (null = self-booked N/A) `· actualSgd · varianceSgd · receiptOk · transactionDate? · reason? · receiptType` (LOCAL_GST|FOREIGN_NO_GST|NO_RECEIPT) · `currency · foreignAmount? · sponsorSgd · chargingCode?/allocJson?` (ITEM allocation). `TravelExpenseLink`: `id` PK · `requestId` FK · `claimId` unique FK · `travellerId?`.

4.3.10 Supporting entities

`TravelRequestVersion`: `requestId` FK · `version · reason` (SUBMISSION|REAPPROVAL) · `approved · snapshot` (JSON) · `createdAt`. `IntegrationMessage`: `requestId` FK · `direction` (OUTBOUND|INBOUND) · `kind` (TMC_HANDOFF|TMC_RESPONSE) · `tmcProviderId? · payload` (canonical JSON) · `createdAt`. *Append-only.* `AuditEvent`: `requestId?` FK · `actorId? · onBehalfOf? · kind` (CREATE|SUBMIT|APPROVE|REJECT|SENDBACK|AMEND|INTEGRATION|TE_LINK|STATUS|COMMENT) · `summary · createdAt`. Comments (§13.21) share this stream with `kind=COMMENT`. *Append-only.* `Attachment`: `requestId` FK · `docType` (Quotation|Conference Invitation|Approval Evidence|Other) · `fileName · sizeKb · contentType? · content: Bytes? · uploadedById? · createdAt`. `Provost`: `id` PK · `schoolId · approverId · name · title · active`; unique `(schoolId, approverId)`, index `schoolId`. `ModuleSettings` (single row id `SETTINGS`) — §5.1. `IntegrationFieldSetting` (`key` PK · `includeTmc · teTreatment`) and `IntegrationGuardSetting` (`key` PK · `enabled · value`) — admin overrides over the §9 contract.

4.4 Status & state vocabularies

ENTITY.FIELD	VALUES (LIFECYCLE ORDER)
`TravelRequest.status`	Draft → Submitted → (Pending RO / Pending Additional / Pending Exception / Pending Provost / Pending DOA) → Approved → (Amendment In Progress ↻) / Cancelled / Expired / Closed; Sent Back → Draft; Withdrawn; *Rejected (legacy, non-target)*.
`TravelRequest.bookingStatus`	Not Sent → Sent to TMC → Received by TMC → Booking In Progress / Traveller Action Required / Partially Booked → Booked → Travel Completed; Booking Failed; Self-Booked; Cancellation Requested → Cancelled.
`TravelExpenseClaim.status`	Draft → Pending Verification → (Pending Exception / Pending RO) → Pending DOA → Approved; Sent Back.
`ApprovalStep.status`	Pending → Approved | SentBack | Skipped (Rejected legacy).
Policy outcome	PASS | WARNING | EXCEPTION | HARD_STOP.

Allowed request transitions (invalid transitions MUST NOT be offered — AC24):

```
Draft→{Submitted,Withdrawn}
Submitted→{PendingAdditional,PendingDOA,PendingException,PendingProvost}
PendingRO→{PendingDOA,PendingException,PendingProvost,SentBack}
PendingAdditional→{PendingException,PendingProvost,PendingDOA,SentBack}
PendingException→{PendingProvost,PendingDOA,SentBack}  PendingProvost→{PendingDOA,SentBack}
PendingDOA→{Approved,SentBack}  Approved→{AmendmentInProgress,Cancelled,Expired,Closed}
AmendmentInProgress→{Approved,SentBack,Cancelled}  SentBack→{Draft,Withdrawn}
Expired→{PendingRO,PendingDOA,Cancelled}  Withdrawn/Cancelled/Closed/Rejected→terminal
```

> **A Travel Request has no Reject** (DD-12): dissent = **Send Back**. `Rejected` is legacy-only.

4.5 Uniqueness & indexes

Unique: `TravelRequest.requestNumber`, `TravelRequest.authorisationNo`, `TravelExpenseClaim.claimNumber`, `TravelExpenseLink.claimId`, `AccommodationEstimate.expenseId`, `ODAEstimate.expenseId`, `Provost(schoolId, approverId)`. Indexed: `Provost.schoolId`. Production SHOULD additionally index `TravelRequest(status)`, `TravelRequest(travellerId)`, `ApprovalStep(approverId, status)`, `AuditEvent(requestId)`, `IntegrationMessage(requestId)` for the queue/dashboard read models (§11.3).

4.6 Data volumes & retention (indicative)

Travel requests scale with staff travel (order 10^3–10^4/yr); each TR has a handful of legs/estimates/allocations/steps and 0–N bookings/segments. Audit and integration messages are the highest-volume append-only tables. Retention follows NTU records policy (travel + finance); versions and audit are retained for the authorisation's lifetime plus the statutory period. **O-R1:** confirm retention periods.

TR-PT-11 Every Travel Request MUST be answerable *as approved* from its immutable version snapshot and audit trail, independent of later edits.

Chapter 5 — Configuration model (the settings and policy values that control behaviour)

In plain terms. Much of how the module behaves is controlled by **settings an administrator can change at any time**, rather than by code — for example whether a Reporting Officer step is required, the claim auto-approval tolerance, the group size limit, and the emergency lead-time window.

Alongside these are the **policy reference values** that come from NTU travel policy and finance: the hotel caps, the overseas-allowance rates and their long-trip taper, the approval amount bands that

decide which DOA tier is needed, and the deviation tolerances.

The tables below list every setting with its default and what it does, and every rate. The key point for the business: changing policy here is a configuration change, not a software release.

5.1 Module Settings (runtime-togglable, AC03 — one `SETTINGS` row)

Maps to OutSystems site properties. A change persists and drives behaviour with no rebuild.

KEY	TYPE	DEFAULT	EFFECT
sameRouteResearch	bool	false	Off → research-charged requests route to the **Research DOA** line.
expenseScope	enum	ALL	`MAJOR_ONLY | ALL` — which expense types are available.
hotelEstimateBasis	enum	LOWER	`QUOTED | CAP | LOWER` — accommodation budget basis (§6.2).
airfareTreatment	enum	IN_TE	`DIRECT_SAP | IN_TE | REFERENCE` — **how** a central airfare line posts; it does **not** decide reimbursability (DD-19, §6.10).
teLinkageMandatory	enum	CONDITIONAL	`YES | NO | CONDITIONAL` — must a claim select an approved TR.
approvalAmountBasis	enum	NET_NTU	`GROSS | NET_NTU | MAJOR_COST | ALLOCATION` — amount used for DOA tiering (§6.2).
groupTravelEnabled	bool	true	Enables group (one-request-many-travellers).
groupMaxTravellers	int	10	Max travellers in a group (TMC group-fare limit); create blocked above.
groupGuestMixAllowed	bool	**false**	Groups are employees-only; guests use a separate request (DD-13).
exceptionApproverRequired	bool	true	On → a policy exception inserts an Exception Approver; off → surfaced to the DOA.
roRequirement	enum	INDIVIDUAL_ONLY	`ALWAYS | INDIVIDUAL_ONLY | NEVER` — RO decision point (DOA always required).
teAutoGrantTolerancePct	int	10	Claim DOA auto-grant tolerance, % over approved reimbursable.
teAutoGrantToleranceAbsSgd	int	500	…absolute SGD cap.
crossBaThresholdSgd	int	500	Non-primary-BA share above this must concur (§6.6).
selfBookingEnabled	bool	true	Allows Self-Booked method + TR selection at claim.
selfApprovalLimitSgd	int	0	Amount ≤ which a traveller-approver may self-approve (0 = never).
attachmentMaxMb	int	7	Max upload per file.
chargingSplitMode	enum	PERCENT	`PERCENT | AMOUNT` entry mode.
declarationText / coiText	string	(text)	Submit declaration + COI statements.
sftpConfig	string	(display)	TMC transport config (display only).
authorisationValidityDays	int	30	TA validity window (§13.3).
bookingDeadlineDays	int	14	Booking deadline window (§13.3).
emergencyLeadDays	int	14	DD-23 standard lead-time window; beyond it the out-of-window justification + confirmation apply.

5.2 Policy reference rates (ECS masters, read-only, effective-dated)

Hotel caps — resolve a nightly cap by city-specific cap first, else the city's **country** cap. City (SGD/night): TYO 350, OSA 300, LON 280, PAR 300, SIN 200. Country fallback: AU 260, CN 240, DE 270, EG 180, FR 290, GB 300, HK 280, IN 180, JP 320, KR 260, MY 150, SG 200, US 330. **ODA rates** (`dailyRateSgd` / `monthlyRateSgd`): AU 125/2250, CN 100/1800, DE 120/2200, EG 90/1600, FR 110/2000, GB 130/2350, HK 115/2050, IN 85/1500, JP 120/2200, KR 110/2000, MY 75/1350, US 140/2500. **ODA duration tiers:** days 1–15 → 100% of daily; 16–31 → 75%; 32+ → `monthlyRate ÷ 30` /day. **Workflow bands** (approvalAmount → DOA tier): BAND-A `0–5,000` → tier 0; BAND-B `5,000.01–20,000` → tier 1; BAND-C `>20,000` → tier 2. **Tolerances:** booking fare `+10%` or `+SGD 200` (lower); date shift `2 days` ; estimate amendment `+10%` or `+SGD 500` . **Travel classes:** TC-ECO Economy (rank 1), TC-PEY Premium Economy (rank 2), TC-BIZ Business (rank 3). **Policy thresholds:** `shortLeadDays 14, longTripDays 14, highEstimateSgd 20000, severityHighSgd 2500, severityMediumSgd 500` .

5.3 Reference-master shapes (the §2 integration seam)

```
Employee{
id,name,title,departmentId,reportingOfficerId?,roles[],isTraveller,travelPolicyGroup,
          workerType?,email?,businessArea?,defaultChargingCode?,travelEligible? }
Department{ id,name,kind:SCHOOL|RESEARCH_INSTITUTE,headId,doaLine[],researchDoaLine? }
ChargingCode{
code,name,type:CC|WBS,subType?,departmentId,companyCode,businessArea,isResearch,
              active?,validTo?,fundingOwnerId?,crossCharge? }
ExpenseType{
id,teCode,name,category,glAccount,gstCode,taxRatePct,preTripApplicable,isMajorCost,
             estimateFormType:AIRFARE|ACCOMMODATION|ODA|STANDARD,varianceThreshold }
TravelClass{ id,name,rank,costIndex }   TravelClassRegisterEntry{
employeeId,entitledClassId,policyGroup,minHours,effectiveFrom,effectiveTo }
WorkflowBand{ id,minSgd,maxSgd?,doaTierIndex,label }   OdaRate{
countryCode,dailyRateSgd,monthlyRateSgd,effectiveFrom }
```

Load-bearing contracts: `Department.doaLine[doaTierIndex]` = the DOA; `Employee.reportingOfficerId` = the RO source; `ChargingCode.{departmentId,businessArea,isResearch,active,crossCharge,fundingOwnerId}` drive routing + the funding block.

5.4 Changing policy / adding a destination

A new destination or rate is **configuration only** (an effective-dated master row); no code change. A Module-Settings change is server-read on the next action. **TR-PT-12** A new destination, cap, ODA rate, workflow band or DOA line MUST be addable through configuration alone and take effect on the next evaluation with no code change or schema migration.

Chapter 6 — Runtime algorithms (the decisions the system makes)

In plain terms. This chapter describes every automatic decision the system makes for a trip: how it works out the entitled travel class and the estimated cost, which policy rules apply, who has to approve, who pays for what, and how it spots a duplicate trip. Each section opens with what the decision achieves in business terms; the numbered steps and code that follow are for the build team.

Every one of these decisions runs on the server and is re-checked when the request is submitted and at each approval, so the result cannot be bypassed from the screen, and the same inputs always give the same answer.

6.1 Auto-estimate derivation — `syncAutoEstimates(id, resetRates)`

Policy-driven ODA + accommodation are **auto-derived one line per overnight city** from the itinerary; the traveller never hand-keys them (airfare stays manual).

1. Take the shared legs (travellerId null), ordered by `seq`.
2. For each leg *i*: `from = leg.arriveDate ?? leg.departDate`; `to = next.departDate ?? next.arriveDate ?? req.endDate`; `nights = round((to − from)/day)`. A city with `nights ≤ 0` yields no stop.
3. **Hotel:** create/update one `AccommodationEstimate` per stop at the **city cap** as the default nightly rate (`budgetedNightly` per `hotelEstimateBasis`), keyed by `(travellerId, legSeq)`. Reset an edited rate back to the cap **only when** `resetRates`.
4. **ODA:** create/update one `ODAEstimate` per city, `days = nights`, amount via the duration tiers (§6.2), keyed by `(travellerId, legSeq)`.
5. Reconcile — update matched keys, create new stops, delete stops that no longer exist. `saveTrip` calls this with `resetRates = (dates/legs signature changed)` — a dates change supersedes a prior manual rate edit. **TR-PT-13** ODA and accommodation MUST be system-derived per city from the itinerary and re-derived whenever the dates or stops change.

6.2 Working out the cost

In plain terms. The system totals the trip cost and separates out what NTU funds (gross cost minus any sponsorship), what will be used to decide the approval tier (the "approval amount"), and the ceiling the travel agency must book within. Hotels are budgeted at the city cap (or the lower of the quote and the cap); an over-cap rate is flagged as an exception and needs a justification. ODA (the daily overseas allowance) steps down for very long trips: full rate for the first 15 days, 75% for days 16–31, and a monthly-allowance rate beyond that — so a normal trip is unaffected and only long secondments taper.

Summary (`computeSummary`): `gross = Σ sgdAmount`; `sponsorship = Σ sponsorSgd`; `major/incidental` split by `expenseType.isMajorCost`; `bookingCeiling = Σ(AIRFARE+ACCOMMODATION)`; `ntuFunded = gross − sponsorship`. **approvalAmount** =

GROSS→gross, MAJOR_COST→major, NET_NTU/ALLOCATION/default→ntuFunded. `approvalAmount` is **always the full committed spend** (settlement-independent, §6.10) and drives DOA tiering. **Accommodation** (`computeAccommodation`): `cap = hotelCap(city)` ; `chargeableNights = max(nights−personalNights,0)` ; `budgetedNightly` = QUOTED→quoted, CAP→cap, LOWER→ `min(quoted, cap||quoted)` ; `overCap = cap>0 && quoted>cap` ; `capVariance = overCap?(quoted−cap)×chargeableNights:0` ; `sgdAmount = budgetedNightly×chargeableNights` ; `outcome = overCap?EXCEPTION:PASS` . **ODA tiered** (`computeOdaTiered({days,dailyRate,monthlyRate})`):

```
t1=min(days,15)            → t1×dailyRate×1.00
t2=max(0,min(days,31)−15)  → t2×dailyRate×0.75
t3=max(0,days−31)          → t3×(monthlyRate÷30)
sgdAmount=round2(t1Amt+t2Amt+t3Amt)
```

6.3 Travel-class entitlement (what cabin the traveller is entitled to)

In plain terms. The entitled cabin (Economy / Premium Economy / Business) is derived from the staff travel-policy register and the length of each flight — it is not chosen on a form. On a multi-city trip the longest flight sets the entitlement. A traveller may request a higher cabin, but that needs a written justification and becomes a policy exception routed to the right approver; a guest above Economy additionally needs a Provost's authorisation.

1. For each flown (non-personal) leg, `entitledForDuration = register.entitledClassId if hours ≥ register.minHours else Economy` (Economy if not in the register).
2. Overall entitlement = the **highest-ranked** per-leg class (multi-city safe).
3. Persist `entitledClassId` + `classBasis` (request, per traveller, per leg).
4. A chosen class above entitlement: (a) require a justification → else hard-stop `CLASS_JUSTIFY` ; (b) raise the `CLASS` exception → Exception Approver (or DOA if off); (c) for a **guest** above Economy, additionally require a selected Provost → else `CLASS_PROVOST` → Provost step (DD-25).

6.4 Policy engine (checking the trip against travel policy)

In plain terms. When a trip is reviewed, the system runs it against every travel-policy rule and labels each result one of four ways: **Pass** (fine), **Warning** (worth noting, but allowed), **Exception** (allowed only with the right approval — e.g. a hotel above the cap), or **Hard stop** (the trip cannot be submitted until it is fixed — e.g. no funding selected). A hard stop blocks submission; an exception adds the appropriate approver to the route. The rules below are held as a maintainable list, so policy can be switched on/off or dated without a code change.

Each rule runs when it is active for the trip's date; a few structural rules are always on. The worst outcome decides what happens: any **Hard stop** blocks submission, any **Exception** adds an approver.

CODE	CATEGORY	OUTCOME	TRIGGER
DATES	Dates	HARD_STOP★	Start/end missing, or end < start.
DATE_PAST	Dates	HARD_STOP	Official **start date in the past** (start-of-today cutoff; same-day OK; drafts exempt).
LEG_OVERLAP	Dates	HARD_STOP★	A leg departs a city **before the previous leg arrives** (when the prior arrive date is set).
ROUTE_GAP	Dates	WARNING	A leg's origin ≠ the previous leg's destination (possible missing connection).
ALLOC	Funding	HARD_STOP★	No allocation, or total ≠ 100%.
FUNDING	Funding	HARD_STOP★	Unknown charging code.
FUNDING_INACTIVE	Funding	HARD_STOP	A closed/inactive CC/WBS.
ESTIMATE	Estimate	HARD_STOP★	No estimate line.
HOTEL_CAP	Accommodation	EXCEPTION	Nightly rate above the city/country cap.
HOTEL_JUSTIFY	Accommodation	HARD_STOP	Above-cap hotel with **empty justification** (inside the HOTEL_CAP loop).
CLASS	Travel class	EXCEPTION	Chosen class above entitlement.
CLASS_JUSTIFY	Travel class	HARD_STOP★	Class upgrade with no justification.
CLASS_PROVOST	Travel class	HARD_STOP★	Guest above Economy with no Provost.
OFFLINE	Booking	EXCEPTION	Offline booking (needs supporting quotation + approval).
LEAD	Lead time	WARNING	Departure within `shortLeadDays` (14).
LONG	Duration	WARNING	Trip > `longTripDays` (14).
HIGH	Estimate	WARNING	Approval amount > `highEstimateSgd` (20,000).
DUPLICATE	Duplicate	WARNING	Overlapping live trip, different destination (§6.11).
HIGH_RISK	High-risk	WARNING	High-risk destination (+ mandatory traveller/approver ack).

★ = `mandatory`. **TR-PT-14** Submission MUST be blocked while any HARD_STOP is unresolved; an EXCEPTION MUST route to the configured approver; a WARNING is advisory.

6.5 Validation rules (field / cross-field / submission)

Trip step (`saveTrip`, blocks non-draft save): required — travel purpose, ≥1 leg with a destination, travel dates, booking arrangement; `end ≥ start`; **official start not in the past**; event dates within the official window; personal extension `end ≥ start` and **not overlapping** the official dates; per leg `arrive ≥ depart`; each leg within the window (widened by a personal extension); legs **chronological** (`n+1 depart ≥ n depart`); **no overlap** (`n+1 depart ≥ n arrive` when the prior arrive is set). **Date gaps between legs are the city stay and are NOT validated. Submission (`submitRequest`):** all hard-stops resolved; no **blocking duplicate** (§6.11); a destination present; a DOA selected when candidates exist; high-risk ack when applicable; emergency gates (§6.15); no

guest-attributed ODA/shared line; valid status transition; self-approval prevented (§6.7). **Claim:** no expense type absent from the approved request; a `NO_RECEIPT > 0` line raises a new exception; a claim over the approved **reimbursable** amount beyond tolerance raises a new exception.

6.6 Approval routing (working out who must approve)

In plain terms. The system builds the list of approvers for a trip automatically, from the trip's cost, funding and policy exceptions — nobody is hard-coded. The route can include, in order:

- an optional extra approver the traveller chose;
- the traveller's Reporting Officer;
- a funding owner;
- a concurring approver for any other business area that is sharing a large cost;
- a policy-exception approver;
- a Provost (for a guest cabin upgrade);
- and always, as the final step, the **DOA** (Delegation-of-Authority holder) whose authority covers the amount.

Crucially, **no one who is travelling on the trip or who raised it can be an approver** — the system routes past them to the next suitable person (this is "separation of duties").

The route is built in this order; at each step the system skips, or escalates past, anyone who is a traveller on the trip or raised it:

1. **Additional Approver** — if `additionalApproverId` set and not excluded.
2. **RO** — per `roRequirement` (ALWAYS, or INDIVIDUAL_ONLY for non-group; NEVER). `resolveRoStep` walks the traveller's reporting line to the first RO not excluded.
3. **Funding Owner** — a charging account with `crossCharge && fundingOwnerId`.
4. **Cross-BA concurrence** — group allocation by business area; for each non-primary BA, if `approvalAmount × share% > crossBaThresholdSgd`, add a concurring approver (the BA's funding owner, else its DOA line), skipping excluded/already-on-route. The primary-BA DOA still owns the request.
5. **Exception Approver** — when `hasException && exceptionApproverRequired`; first `EXCEPTION_APPROVER` not excluded.
6. **Provost** — a guest class upgrade with a selected Provost (DD-25).
7. **DOA (always)** — `line = isResearch && !sameRouteResearch ? researchDoaLine : doaLine`; `tier = bandForAmount(approvalAmount).doaTierIndex`; use `selectedDoaId` when set & not excluded, else `pickDoa` walks the line from the tier escalating past excluded holders; role = `RESEARCH_DOA` or `DOA`. `statusForStep(firstPending)` sets the Pending-* status. **Emergency** builds the *same* route (no step skipped) and annotates each step "retrospective ratification (book-first, DD-23)". **TR-PT-15** Every request MUST terminate at a DOA resolved from the highest-charging department's DOA line at the amount-band tier, and MUST NOT auto-select it — the requestor chooses from the eligible candidates.

6.7 Self-approval prevention (DD-28)

No one in `selfApprovalSet` (travellers + requestor) may be an approver: single-holder steps escalate up the relevant line, multi-candidate steps pick the first eligible. Approver-only personas cannot originate a request (§8.3). **TR-PT-16** The solution MUST prevent any person approving a request they travel on or raised, by routing past them to an alternate approver, and MUST prevent an approver-only persona from originating a request.

6.8 Charging roll-up

`chargingMode` REQUEST (one split) or LINE (per-line `chargingCode` / `allocJson`). The request-level allocation driving routing is the **roll-up** of `net × %` over every line × split. `highestChargingDept` = the department bearing the largest share (tie-break to the requestor's department) → the DOA owner and cross-BA basis. "Auto-allocate by Workday department" fills each line from the attributed traveller's `defaultChargingCode`.

6.9 Authorisation issue

On the final DOA approval: set `status = Approved`; allocate `authorisationNo = TA-YYYY-NNNNNN`; set `bookingDeadline` (+`bookingDeadlineDays`) and `authorisationExpiry` (+`authorisationValidityDays`); snapshot an approved `TravelRequestVersion`; for a TMC arrangement, auto-hand-off to the TMC (DD-26, §6.12).

6.10 Who pays for what — settlement & reimbursement (DD-19)

In plain terms. "Settlement" is simply *who fronts the money* for each cost, which decides whether the employee can later claim it back. The rule per cost type:

- **Airfare** booked through the travel agency is paid centrally by NTU (the Central Travel Account, CTA), so it is shown on the claim for reference but is **not** reimbursed to the employee (they never paid it). A flight the traveller books themselves *is* reimbursable.
- **Hotel** is the requestor's choice per trip — pay centrally (CTA), or let the traveller pay and claim it back.
- **ODA and other costs** are always paid by the traveller and claimed.

This decision is kept in one place so the booking instruction, the claim and the approval all stay consistent. It does **not** change how much is approved or which budget bears the cost — only who is out of pocket.

The one rule (used everywhere) for a cost line's settlement:

```
AIRFARE       → CENTRAL if isTmcArrangement(bookingArrangement) else EMPLOYEE   (self-booked flight = reimbursable)
ACCOMMODATION → CENTRAL if hotelSettlement==='CTA' (TMC arrangement) else EMPLOYEE
ODA / OTHER   → EMPLOYEE (always)
```

`isReimbursable = settlement===EMPLOYEE`. **Reads:** (a) TMC payload — each hotel segment `billTo = CTA|TRAVELLER`; (b) claim treatment — CENTRAL → `REFERENCE` (shown, not reimbursed), EMPLOYEE → `EDITABLE`; (c) DD-08 tolerance — compares the **reimbursable** portion on **both** sides (§6.14). `airfareTreatment` only governs *how* a central airfare line posts, never reimbursability. The hotel toggle applies to individual + group; CTA offered only for a TMC arrangement. Settlement is **separate from authorisation and cost ownership** — the approval amount is the full committed spend and the CC/WBS allocation is unchanged. **TR-PT-17** A claim line MUST be reimbursable if and only if the employee fronted it; centrally-settled lines (CTA/TMC-billed) MUST be shown for reconciliation but excluded from the reimbursable total.

CTA charge posting & reconciliation (DD-30). ECS's existing posting programme performs the GL posting of central (CTA) charges; the one input it lacks is the **clearing account** per CTA. A small **CTA master** supplies it — a posting-only lookup keyed by the `ctaAccountId` the card/TMC feed reports on each charge (never derived), mapping it to its clearing GL account (different CTA accounts → different clearing accounts). On the claim the CTA-settled lines are **Reference** and auto-deducted from the reimbursable total (above). A read-only **CTA Charges** listing groups the captured charges by clearing account (with CSV export) — a one-sided view for finance to reconcile against the provider statement outside ECS; ECS performs no matching or posting there. **Guest bookings are CTA-settled too** (airfare + accommodation): their charges are captured and listed for reconciliation, but a guest files **no TE claim** (DD-21), so guest charges never appear on a claim. Guest charges may lodge to the **same CTA account as employees or a separate guest CTA account** — the account is feed-reported, so the master simply holds whichever the TMC uses. **TR-PT-28** Each central-settled charge MUST carry the feed-reported CTA account; ECS MUST resolve its clearing account from a configurable CTA master (distinct CTA accounts resolving to distinct clearing accounts) and MUST NOT derive the account itself. Guest bookings MUST be CTA-captured for reconciliation without a TE claim, and the CTA account MAY be shared with employees or dedicated to guests.

6.11 Catching duplicate or clashing trips (DD-14)

In plain terms. A traveller cannot be in two places at once, and the University does not want two live authorisations (and two sets of bookings) for the same trip. So the system looks at the traveller's other **active** requests (anything submitted, awaiting approval, approved or being amended) and compares dates and destination:

- **Same dates and same destination** — a true duplicate: the new request is **blocked** from submission until the original is cancelled or amended.
- **Overlapping dates, different destination** — a possible double-booking of the person's time: the system shows a **warning**, not a block.

This check is shown **early** — on every editable step as soon as the dates are entered — so the traveller is not surprised at the end; the block at submission is the final safeguard.

Technically: among the traveller's other live requests with overlapping official dates, same country (and same city where both are set) is a true duplicate → hard block; a different destination → warning. **TR-PT-18** The solution MUST block a second live authorisation for the same traveller + overlapping

dates + same destination, surfaced from the first editable step, and MUST warn on an overlapping different-destination trip.

6.12 TMC hand-off, group fan-out & multi-PNR

Hand-off (`handoffToTmc`): build the canonical outbound payload (§9.1) honouring the Integration Contract; send only when the outbound guards hold (Approved · Not Sent · TMC-served method · TA set); emergency sends on submit (DD-26); **idempotent** (never re-send an in-flight/booked request). **Group fan-out:** one instruction per traveller, each with that traveller's effective itinerary/class/nights/share. **Multi-PNR:** a `≥3` -air-segment multi-city trip is split into PNR-A (first half + hotel) and PNR-B on inbound; each PNR is its own `TravelBooking` .

6.13 Inbound — TA-keyed retrieval & deviation (`receiveBooking` , `refreshBookingsFromTmc` , `bookingsForAuthorisation`)

The claimant creates the claim **from the TR**; the TMC exposes retrieval **by the TA number**, returning the **full PNR set** (not a PNR-by-PNR lookup). `bookingsForAuthorisation(req)` → `{authorisationNo, pnrCount, bookings[{pnr,ticketNo,providerId,travellerId,status,fareSgd,taxesSgd,feesSgd,hotelRateSgd,totalSgd,segments[{type,origin,dest,departDate,arriveDate,bookedClassId,amount}]}], totalFareSgd}` . **Push-primary** (receiveBooking stores bookings against the TR so the claim pre-fills with zero PNR entry) + a **pull-by-TA refresh** (idempotent). A self-booked trip returns an empty set. **Deviation:** booked aggregate airfare vs approved + tolerance (`+10%` / `+SGD 200` , lower) → material deviation → re-approval. `legPnrMap(req)` maps each AIR leg to its PNR (origin+dest + date) for per-leg provenance on the TR (§11). **TR-PT-19** Booking retrieval MUST be keyed by the Travel Authorisation (returning all PNRs under it), not the PNR; captured actuals MUST include booked fares and booked dates.

6.14 Seeding the expense claim & auto-approving the DOA when it matches (DD-08)

In plain terms. After the trip, the employee's expense claim is pre-filled from the approved travel request, so they are not re-typing it.

If the claim **matches what was approved** — within a small tolerance, no new cost types, no new policy exception, same funding — the system **auto-approves the DOA step**: there is no need to ask the DOA again for something already authorised. If instead the claim exceeds the approved amount beyond tolerance, adds a new cost type or exception, or moves to a different funding account, it is **not** auto-approved and routes to the right DOA.

Centrally-paid costs (airfare, CTA hotel) appear on the claim for reference but are not part of what is reimbursed, and both the claim and the approved figure are compared on the same (reimbursable) basis so the tolerance is fair.

Pre-fill detail: the header is mostly read-only; each cost line is marked *Reference* (shown, not reimbursed) when it was centrally paid, otherwise *Editable*; lines are filtered to this claimant; shared accommodation is costed on the claimant's own nights; amounts already claimed earlier are netted

off. **Route:** Verifier → (Exception if a new exception) → (RO per `roRequirement`) → DOA (always; from the claim's effective charging dept + band). **Auto-grant** (`doaAutoGrant`) — all must hold: TR Approved/Closed with a TA; `claimTotal − approvedAmount ≤ tolerance` (% and SGD); no new expense type; no new exception; **unchanged charging DOA** (a changed account routes to the new account's DOA, never auto-granted). Scope: `claimTotal = Σ non-REFERENCE lines − corpCard − personal`; `approvedAmount = Σ reimbursable (EMPLOYEE) approved expenses net of sponsorship` — **like-for-like**, central airfare + CTA hotel excluded from both sides.

6.15 Emergency lane (DD-23)

Book-first/raise-later, bounded & audited. Submit gates: a named **authorising officer** (not a traveller) + the authorising **e-mail attached** (mandatory evidence); **agent-assisted/offline TMC only**; destination mandatory; policy hard-stops still apply; soft-gated by `emergencyLeadDays` — outside the window a detailed justification (≥ min length) + an explicit confirmation are required and the request is flagged *Outside standard window*. The **full route still runs** (same approvers, ratified retrospectively, §6.6); on submit the instruction is auto-handed to the TMC with an `EMERGENCY_BOOK_FIRST` indicator. **TR-PT-20** The emergency lane MUST change only the *timing* of approval, never the set of approvers, and MUST require a named authoriser + attached authorising e-mail; it MUST be available only for agent-assisted/offline TMC bookings.

Chapter 7 — Lifecycle & change governance (how a request moves and changes over time)

In plain terms. This chapter follows a request through its life — from draft, through approval, to an issued authorisation, booking, trip and claim — and sets out how changes are handled once it is approved. The guiding rules are:

- an approver who disagrees **sends the request back** to be revised rather than rejecting it;
- a **material change** to an approved trip re-runs the full approval and re-issues the authorisation, so what is booked always matches what was approved;
- a **failed booking** is recovered in a controlled way (retry, amend-and-re-route, or cancel-and-re-raise) without ever leaving two live authorisations;
- **cancellation** goes through the travel agency once anything has been booked.

Every change is recorded in the audit trail, and each submitted/approved state is frozen as a version for the record.

7.1 Request lifecycle

Draft → Submit (route built) → approvals (each `approveStep` advances or finalises) → **Approved + TA** → TMC hand-off → Booked → Travel Completed → TE claim. Dissent at any step = **Send Back** (→ Sent Back → Draft) with a reason; there is no Reject (DD-12).

7.2 Amendment

A material change to an Approved request moves it to **Amendment In Progress**, re-derives the route and re-approves, issuing a revised TA. The amendment threshold is `estimateAmend` tolerance (`+10%` / `+SGD 500`).

7.3 Failed-booking recovery (§6.4 / DD-15)

The OBT reports booking **status**, not an approval. Not issued within the window → **Booking Failed**; recovery (gated by the window): **re-send** (retry), **amend & re-route** (reopen → Amendment In Progress, single-TA path), **cancel & re-raise** (original terminal, frees the overlap under DD-14). A fare over tolerance routes to re-approval.

7.4 Cancellation (DD-22)

Before hand-off → local withdraw/cancel. After transmit/book → via the TMC (`cancelRequest` → Cancellation Requested → `receiveCancellation` → Cancelled). This + the duplicate guard prevents a double-booking on a later re-raise.

7.5 Versioning & audit

Every submission/approval snapshots an immutable `TravelRequestVersion` (`currentVersion++`). Every field/workflow/integration action and every comment is an append-only `AuditEvent` . **TR-PT-21** Amendment of an approved request MUST re-run the full approval route and re-issue the TA; the prior authorisation version MUST be retained.

Chapter 8 — Security & access control (who can do what)

In plain terms. Three separate questions decide what a person can do: *can they raise a request, and for whom?*, *can they edit this draft?*, and *can they approve this step?* Holding one of these never grants another — an administrator can help raise or edit requests but is **not** an approver, and an approver cannot raise a request they might then approve. Every one of these checks runs on the server, so the screen can never be used to get around them.

8.1 Three mechanisms, kept separate

(1) **Creation remit** — may this persona originate a request, and for whom. (2) **Edit remit** — may this persona edit a draft. (3) **Approver entitlement** — may this persona act on *this* step. They are independent; holding one never implies another.

8.2 Identity modes

Prototype: a persona cookie (`ecs_persona` + `ecs_role`) drives `currentPersonaId()` / `currentRoleId()` ; the switcher groups personas by stage (REQUEST / CLAIM / ADMIN). **Production MUST** replace this with ECS SSO/identity; the persona id becomes the authenticated employee id and

roles come from ECS. **TR-PT-22** Production MUST derive identity and roles from ECS authentication, not a client-set value.

8.3 Creation remit — `creatableTravellerIds(persona)` / `canCreateTravelRequest`

A persona may raise for: **themselves** (if a traveller); **principals who delegated** to them (`EcsDelegation.canCreateTravelRequestFor`); for a **Travel Requestor/PA** — travellers in their department; for the **Travel Administrator** — any traveller (agent-assisted). **No catch-all fallback:** an approver-only persona (DOA/Research DOA/Exception), System Administrator or Finance viewer gets an **empty set** and cannot originate a request (DD-28 / separation of duties).

8.4 Edit remit — `canEditRequest(persona, req)`

True for: Travel/System Administrator; the creator (`requestorId`); the traveller themselves; a delegate for the traveller; a Travel Requestor in the traveller's department. **Explicit remit only** — read-only roles and approvers are excluded.

8.5 Approver entitlement & separation of duties — `canActOnStep` , `isApproverPersona`

An approval runs only when `approverId && approverId === persona` — the acting persona **is** the step's assigned approver. **A Travel/System Administrator does NOT get an approval bypass** — they may approve only when they independently hold the role and are the assigned approver. A step with **no resolved approver is not open** to any persona (closes the null-approver hole). The RO/DOA dashboard is gated by `isApproverPersona` (admins are not approvers). **Self-approval** is prevented in routing (§6.7), not just the UI.

8.6 Lifecycle guards — `actionAllowed(action, status)`

Every mutating action checks a valid **source state** before running: `submit` ∈ {Draft, Sent Back}; `approve` / `sendBack` ∈ {Pending RO/Additional/Exception/DOA, Amendment In Progress}; `withdraw` ∈ {Draft, Sent Back, Pending*}; `reopen` ∈ {Sent Back}; `cancel` ∈ {Approved, Amendment In Progress, Expired}. There is **no reject** action.

8.7 Data scope & audit

The dashboard is **traveller-scoped** (a persona sees their own + delegated + department requests; administrators see all; the approver queue shows steps assigned to the acting persona). Finance is read-only oversight. Every action writes an `AuditEvent` ; audit/integration are append-only. **TR-PT-23** Every mutating action MUST re-check the source state and the acting remit server-side; the client MUST NOT be trusted to have gated it.

Chapter 9 — Integration & external interfaces (talking to other systems)

In plain terms. The module reads reference data from ECS/HR/DOA/Finance and sends a governed booking instruction to the travel agency, then receives the booking back (the PNRs and fares). It also feeds the approved allocation to finance, raises visa-letter and notification messages, and seeds the expense claim. Every message is standardised and logged. The booking information comes back **keyed by the Travel Authorisation** — i.e. "give me everything booked under this approval" — so a multi-city trip split across several airline references is retrieved in one go.

9.1 Principles

One governed boundary per external system; the module **reads** masters and **sends** governed, field-filtered instructions; inbound responses are status, never authority. All outbound/inbound payloads are canonical JSON persisted as `IntegrationMessage` rows. The Integration Contract is **declarative and admin-overridable**.

9.2 System inventory

SYSTEM	DIRECTION	FLOW
ECS / HR identity, DOA, Workday	Read	Employees, reporting lines, DOA lines + bands, home-dept cost object.
SAP Finance	Read + Write*	CC/WBS, GL, GST, IO (read); approved allocation → GL (*via claim/finance IF); DIRECT_SAP airfare posting.
Policy masters / FX / locations	Read	Caps, ODA, class register; effective-dated FX; IATA airport master.
TMC / OBT	Outbound + Inbound	Governed booking instruction (field-filtered; one per traveller for a group) ⇄ booking status + **PNR set keyed by TA**.
Immigration / Visa	Outbound	Daily visa-letter batch for approved visa-flagged trips.
Notifications	Outbound	Approval/action/booking/deadline alerts.
ECS TE Claim	Outbound (seed)	Approved TR pre-populates the claim; reconciles to the TA; DOA auto-granted within tolerance.

9.3 Outbound payload (Send to TMC) — `CONTRACT_FIELDS`

Core (always): `authorisationNumber, travellerRef, bookingMethod`. **Toggleable (default on):** group `travellers`, `bookingDeadline`, `costCeiling`, `approvedExceptions`, `tripPurpose`, `eventDates`, `approvalValidity`, `airSegments` (origin/dest/depart+arrive), `airApprovedClass`, `hotelSegments` **(city/nights/cap +** `billTo` **CTA|TRAVELLER)**, `charging` (CC/WBS/BA), `approvalMetadata`, `travellerCategory` +contact, `personalIndicator`, `guestProfile` (PDPA secure-link, guest-only). Emergency adds the `EMERGENCY_BOOK_FIRST` indicator. **Guards (** `CONTRACT_GUARDS` **):** OUTBOUND — status=Approved ∧ bookingStatus=Not Sent ∧ TMC-served method ∧ TA set. INBOUND — in-flight ∧ TA set. TE_SYNC — status ∈ {Approved, Closed} ∧ TA set.

9.4 Inbound record shape (TA-keyed)

See §6.13 — `bookingsForAuthorisation` return shape. Group fan-out returns one booking per traveller; multi-city splits across PNRs; self-booked returns an empty set. Deviation is checked on the aggregate airfare.

9.5 Interface requirements

TR-PT-24 The TMC interface MUST accept a field-filtered governed instruction and expose booking retrieval by TA (returning all PNRs), and MUST support a group fan-out (one instruction per traveller). **TR-PT-25** Guest passport/profile MUST be collected by the TMC via a secure link; ECS MUST store only guest name/email/organisation (PDPA, DD-21). **TR-PT-26** All external hand-offs MUST be idempotent and fully audited as `IntegrationMessage` rows.

Chapter 10 — Service & action catalogue (the operations the system performs)

In plain terms. This chapter is the catalogue of every operation the system can carry out — creating a draft, saving the trip, adding costs, submitting, approving, sending to the agency, receiving the booking, creating the claim, and so on. For each one it records what triggers it, who is allowed to run it, and what it changes. There is no separate "API" to learn: each operation is a single, self-contained action that checks the rules before it does anything. A business reader can treat the tables as a list of "what the system does and who may do it"; the `monospace` names are the labels the build team uses.

10.1 Conventions

No REST layer in the prototype — **mutations are server actions** (`"use server"`), **reads** are server-component loaders (§11.3). Each action: resolves persona → checks source state + remit → transacts → audits → `revalidatePath` / `redirect(?error=)` . Production MAY expose these as Service Actions / REST.

10.2 Integration seam (ECS services — the boundary to wire)

`EcsIdentity` (employee, department, reportingOfficer, hasRole), `EcsRoles` (employeesWithRole), `EcsDelegation` (canCreateTravelRequestFor), `EcsReference` (expenseType, travelPurpose, country/city/airport, travelClass, currencies/fx, flightHours, highRisk, rejectionReasons), `EcsFx` (sgdPerUnit/toSgd, effective-dated), `EcsPolicy` (hotelCap, odaRate/odaMonthlyRate, tolerances, ODA-tier constants), `EcsTravelClassRegister` (entry, entitledForDuration/Itinerary, rank), `EcsCharging` (code), `EcsWorkflow` (bandForAmount). Contracts in §5.3.

10.3 Server actions (pre-trip) — contract

ACTION	GUARDS	EFFECT
createDraft / createGuestDraft / createGroupDraft	creation remit (§8.3)	New draft (individual / guest / group employees-only).
addGroupTraveller / removeGroupTraveller	edit remit	Manage group (≤ `groupMaxTravellers`).
saveTrip	edit remit; §6.5 validation	Persist trip/dates/itinerary; re-derive auto estimates (§6.1).
addLeg / removeLeg / moveLeg	edit remit	Itinerary edits.
addAirfare / addAccommodation / addOda / addOther	edit remit; entitlement-scoped	Add estimate lines (no ODA for guests).
setHotelSettlement	edit remit; TMC-arrangement for CTA	DD-19 CTA/Individual (§6.10).
updateAccommodationRate	edit remit	Inline hotel rate (above-cap justification).
deleteExpense	edit remit	Remove a manual line.
saveCharging	edit remit	Allocation split (=100%).
runPolicy	edit remit	Re-run engine; persist checks.
submitRequest	§6.5 submission gates + duplicate + self-approval	Build route; set Pending-*.
approveStep	`canActOnStep` (assigned approver)	Advance or finalise (issue TA + auto hand-off).
sendBack	assigned approver	→ Sent Back (reason).
withdrawRequest / reopenDraft	edit remit + source state	Draft/sent-back lifecycle.
cancelRequest / receiveCancellation	source state	§34 cancellation (TMC-mediated after hand-off).
handoffToTmc	outbound guards; idempotent	Transmit the instruction.
receiveBooking	inbound guards; idempotent	Ingest booking(s) — fan-out + multi-PNR + segments + deviation.
refreshBookingsFromTmc	TA set; idempotent	TA-keyed pull.
setBookingTmcStatus / markSelfBooked	source state	Lifecycle / self-booked.
postAirfareToSap	DIRECT_SAP + booked	SAP airfare posting.
resendToTmc / amendAfterFailedBooking	failed state	§6.4 recovery.
runVisaLetterBatch	—	Daily visa notification.

ACTION	GUARDS	EFFECT
addComment / addAttachment	edit/comment remit; ≤ `attachmentMaxMb`	Comment feed / evidence.
copyToNewRequest	creation remit	Clone to a new draft.

TE claim: `createClaimFromRequest(requestId)` (TE_SYNC guards; prepopulate), `updateClaimActuals`, `submitClaim` (build claim route + auto-grant), `approveClaimStep`, `sendBackClaim`, `updateClaimHeader`, `addClaimExpense`, `removeClaimExpense`, `saveClaimCharging`.

10.4 Key call chains

Submit: `submitRequest` → `evaluatePolicies` + `findDuplicateRequests` → `computeSummary` → `buildRoute` → persist steps + status. Final approval: `approveStep` → issue TA → `handoffToTmc` → `assembleOutbound` / `buildOutbound`. Inbound: `receiveBooking` → `simulateInbound` (adapter) → split → persist bookings/segments → deviation. Claim: `createClaimFromRequest` → `prepopulateClaim` (settlement treatment) → `buildClaimRoute` → `doaAutoGrant`.

10.5 Separation-of-duties / integrity checklist (where enforced)

CHECK	WHERE
No create by approver-only persona	`creatableTravellerIds` (empty set)
No self-approval	`selfApprovalSet` in `buildRoute`
No admin approval bypass	`canActOnStep` (`approverId===persona`)
No null-approver step open	`canActOnStep` (`!!approverId`)
Valid source state	`actionAllowed`
100% allocation	`ALLOC` hard-stop
Hard-stop blocks submit	`hasHardStop` in `submitRequest`
Like-for-like tolerance	`doaAutoGrant` reimbursable scope
Idempotent hand-off/inbound	`handoffToTmc` / `receiveBooking`

Chapter 11 — Screen catalogue (the screens people use)

In plain terms. This chapter lists every screen in the module and what each one is for, what the user enters or sees, and which rules it applies. The traveller's journey runs through a six-step wizard — **Traveller** → **Trip** → **Estimates** → **Charging** → **Policy Review** → **Submit** — and each step saves as you go, so a half-finished request can be parked as a draft and the full checks are applied only at submission. Alongside the wizard are the approver and finance dashboards, the booking screen, the

claim, and the admin/configuration screens. The table is a quick map of where each piece of the process happens.

11.1 Global shell

Persona switcher (role + person), role-gated left nav, request/claim/admin stages. Wizard stepper: **Traveller** → **Trip** → **Estimates** → **Charging** → **Policy Review** → **Submit** (soft-save; validation deferred to Submit).

11.2 Route catalogue

ROUTE	SCREEN	FIELDS / ACTIONS / VALIDATION
/dashboard	Dashboard	Traveller-scoped; grouped by drafts/pending/approved-unbooked/completed-unclaimed; search/type/status; S/E flag.
/requests/new	Create	Individual/group/guest; creation-remit gated.
/requests/[id]	Traveller	Traveller + delegate/PA context.
/requests/[id]/trip	Trip	Purpose; booking arrangement + method; resolved TMC; dates (official/event/personal); itinerary builder; visa flag; **upfront duplicate banner**; `saveTrip` (§6.5).
/requests/[id]/estimates	Estimates	Auto per-city ODA+hotel; **hotel-settlement toggle** (TMC-gated); add airfare/accom/ODA/other; inline hotel rate; duplicate banner.
/requests/[id]/charging	Charging	Request/line split (=100%); auto-by-department; PERCENT/AMOUNT.
/requests/[id]/policy	Policy Review	Run engine; outcomes; re-run; duplicate banner.
/requests/[id]/review	Review & Submit	One-page summary (Trip incl. **PNR provenance** once booked; Charging; Estimates; route preview + DOA selector); declaration + COI; high-risk ack; emergency panel; hard-stop/blocking-duplicate disable Submit; `submitRequest`.
/requests/[id]/approve	Approver/DOA	One-page Approval Summary; Approve / Send Back.
/requests/[id]/booking	Booking & TMC	Send/receive/set-status/self-booked/re-send/amend/cancel; per-PNR cards; ↻ **Refresh from TMC**; SAP posting.
/requests/[id]/pack · /details	Pack / Full details	Printable pack; full detail cards.
/claims/[id]	TE claim	Pre-filled from TR; actuals; would-auto-grant; submit.
/approvals	RO/DOA dashboard	Travel-Requests/Expense-Claims tabs; per-approver counters; exception shading.
/workbench · /finance · /reports	Admin / Finance / Reporting	Overrides/ops; read-only oversight; aggregates.
/visa-letters · /notifications · /integration · /config-viewer · /settings · /setup/provosts	Admin	Visa batch; notifications; contract editor + payload preview; reference viewer; Module Settings (SysAdmin); Provost list.

11.3 Data loaders / read models

`loadRequest(id)` eager-loads the full aggregate (travellers, legs, expenses+accommodation/oda, allocations, policyChecks, approvalSteps, bookings+segments, deviations, teLinks, claims, messages, auditEvents, attachments). `listRequests()` is the dashboard read model. Production SHOULD back the dashboard/queue with the indexes in §4.5.

Chapter 12 — Reference data & example scenarios (the data behind the prototype)

In plain terms. The prototype runs on real, representative data so the behaviour can be shown to business users, not just described. Two kinds of data are listed here.

First, the **reference data** — the people, departments, funding accounts, caps, allowance rates, travel-class policy, airports and so on. In production this comes from the existing ECS/HR/DOA/Finance systems; the prototype holds its own copy only so it can run on its own.

Second, a set of **worked example trips** (28 of them) pre-loaded so every feature can be demonstrated end to end — a standard approved-and-claimed trip, a research trip, an over-cap hotel, a group trip, a multi-city trip booked across two airline references, an emergency "book-first" trip, a long secondment that exercises the allowance taper, guest trips, and so on. These examples are the basis for the demonstration and the acceptance tests in Chapter 13.

12.1 Reference-data inventory (`src/data/*` , read through §2)

`employees` , `organisation` (departments/entity), `charging` (CC/WBS), `expenseTypes` , `travelPurposes` , `locations` (countries/cities/airports + SIN flight-hours), `policyRates` (hotel caps + ODA + tiers), `travelClass` + `travelClassRegister` , `workflow` (bands), `fxRates` , `tolerances` , `delegations` , `rejectionReasons` , `highRiskDestinations` , `tmcProviders` , `personaRoles` , `visaLetter` . These are the masters a production build wires to live ECS.

12.2 Seed scenarios (demo, 28) — lazy-seeded when empty

The demo data loads 28 worked trips, each exercising a different feature:

SCENARIO	WHAT IT DEMONSTRATES
S01	Standard trip — approved, booked, claimed
S02	Research trip — routes to the Research DOA
S03	Above-cap hotel — exception, Pending Exception
S11	Group conference — a PA raises three travellers
S12	Group amendment — loses a traveller → reapproval
S14	Multi-leg (Tokyo + Osaka) with a personal day
S15	Travel-class upgrade exception
S16	Multi-country (London + Paris)
S18	Emergency book-first (Seoul)
S19	A draft parked mid-entry
S20–S23	Pending-stage requests across trip types
S22	Multi-city pending
S23	One-way pending
S24	Long-term secondment (45 nights → ODA tiers)
S25	Guest, Seoul — pending
S26	Guest, London — booked
S27	Emergency-eligible draft at submit
S28	**Multi-city booked across two PNRs** (per-leg PNR provenance)

The seed asserts no hard-stop at submit (bad seed data fails the build) and exempts historical-dated fixtures from `DATE_PAST`.

Chapter 13 — Verification & acceptance (how to confirm it works)

In plain terms. This chapter sets out how to check that a build does what this document describes. The approach is behaviour-first: because every rule is already working in the prototype against real policy data, acceptance means a supplier's build is confirmed when it reproduces those behaviours. The table below is a plain checklist a business user or tester can walk through — e.g. "an over-cap hotel is capped and needs a justification", "a true duplicate blocks submission", "a matching claim is auto-approved".

One honest point to note: the prototype proves its behaviour by demonstration and worked examples, not by an automated test suite, so a production build should add one (see the requirement at the end of this chapter).

13.1 Principles

Acceptance is behaviour-first: each rule/algorithm is demonstrable in the running prototype against seeded policy data. The prototype is the executable specification; a respondent's build is accepted when it reproduces the behaviours below.

13.2 Acceptance test catalogue (illustrative)

AREA	ACCEPTANCE
Entitlement	Multi-city class = highest per-leg entitlement; upgrade without justification is a hard-stop.
Cost	ODA follows the duration tiers (e.g. 45 nights = 15×100% + 16×75% + 14×monthly÷30); above-cap hotel is capped + exception + mandatory justification.
Policy	Each rule in §6.4 fires on its trigger; hard-stops disable Submit.
Validation	Past start, leg overlap, route gap, non-100% allocation, inactive account each behave per §6.5.
Routing	Route = Additional?→RO?→Funding/Cross-BA?→Exception?→Provost?→DOA; self-approver escalated; approver-only persona cannot create.
Settlement	Airfare + CTA hotel are REFERENCE; individual hotel + ODA reimbursable; self-booked airfare reimbursable; approval amount unchanged.
Duplicate	True duplicate blocks submit (surfaced upfront); date clash warns.
TMC	TA-keyed retrieval returns all PNRs; multi-city splits into 2 PNRs; hand-off/inbound idempotent; booked arrive dates captured.
Claim	DOA auto-granted within tolerance; changed account routes to the new DOA; reimbursable scope like-for-like.

13.3 Automated test inventory

The prototype has **no automated test suite** (verification is behavioural, via seeded scenarios). **TR-PT-27** A production build SHOULD provide an automated acceptance suite covering the §13.2 catalogue (entitlement, cost tiers, policy outcomes, validation, routing/SoD, settlement, duplicates, TMC, claim auto-grant).

Chapter 14 — RFP / production considerations

14.1 Using this document

Chapters 4–6 and 9–11 are the build contract; Chapter 14.2 is where the prototype simplifies; Appendix D records binding design decisions; Appendix E indexes the derived requirements.

14.2 Prototype-to-production gap register

#	AREA	PROTOTYPE	PRODUCTION REQUIREMENT
G-01	Identity	Persona cookie switcher	ECS SSO/identity; roles from ECS (TR-PT-22).
G-02	TMC	Inbound simulated by an adapter	Real TMC/OBT interface (outbound + TA-keyed inbound), idempotent, audited (TR-PT-24/26).
G-03	Money	SGD `Float`	Exact decimal (TR-PT-04).
G-04	Storage	SQLite; attachments in-row	Production RDBMS; document store (bytes out of the row).
G-05	API	Server actions only	Expose Service Actions / REST as required.
G-06	Structured values	JSON-in-string	JSON/JSONB or normalised child tables.
G-07	Versioning	Snapshot-on-submit/approve	Confirm full field-level version history + as-at enquiry (TR-PT-09/11).
G-08	SAP / visa / notifications	Modelled as actions/audit	Wire to SAP (GL + airfare posting), immigration batch, notification channel.
G-09	Tests	Behavioural only	Automated acceptance suite (TR-PT-27).
G-10	FX	Monthly table	Live effective-dated FX, re-locked at submission (TR-PT-06).

14.3 Indicative non-functional requirements (University to complete `[ ]`)

- **Performance:** policy+route evaluation p95 `[ ]` ms; dashboard/queue load p95 `[ ]`; concurrent users `[ ]`.
- **Availability / RPO / RTO:** `[ ]`.
- **Volumes:** requests/yr `[ ]`; peak group size (TMC limit `10`); audit/integration retention `[ ]` (O-R1).
- **Security:** SSO, role-based access, separation of duties (Ch 8), full audit; PDPA for guest data (TR-PT-25).
- **Accessibility / i18n:** `[ ]` (NTU standards).
- **Integration SLAs:** TMC hand-off / inbound latency `[ ]`; failure/retry semantics.

14.4 Suggested deliverables & acceptance

Source + migrations + seed; the §13.2 acceptance suite; the integration adapters (TMC, SAP, HR/DOA, visa, notifications); the configuration screens; and a demonstration reproducing the seeded scenarios (Ch 12).

14.5 Information to request from respondents

TMC/OBT interface capability (TA-keyed retrieval, group fan-out, secure guest-profile link); SAP GL/airfare posting interface; HR/DOA/Workday read interfaces; identity/SSO integration; data-migration approach; the proposed test strategy against Ch 13.

Appendix A — Glossary

Business acronyms.

- **TR** — Travel Request
- **TA** — Travel Authorisation (the dated, amount-bound approval)
- **DOA** — Delegation of Authority (the person authorised to approve a given amount)
- **RO** — Reporting Officer
- **ODA** — Overseas Daily Allowance
- **TMC** — Travel Management Company (the travel agency)
- **OBT** — Online Booking Tool
- **PNR** — Passenger Name Record (an airline booking reference)
- **CTA** — Central Travel Account (NTU pays the agency centrally)
- **CC/WBS** — Cost Centre / Work Breakdown Structure (the funding account)
- **BA** — Business Area (a finance grouping)
- **TE** — Travel Expense (the post-trip claim)
- **SoD** — Separation of Duties
- **PDPA** — Personal Data Protection Act
- **GL/GST** — General Ledger / Goods & Services Tax

Technical terms in plain words.

- **Entity** — a type of record the system stores (like a table in a spreadsheet); a *static entity* is a fixed reference list.
- **Application Object (AO)** — the unit OutSystems uses to size/licence a build (Appendix F).
- **Server-side / server-authoritative** — the rule runs on the system, not in the browser, so it cannot be bypassed from the screen.
- **Deterministic** — the same inputs always produce the same result.
- **Idempotent** — safe to run twice; doing it again has no extra effect (e.g. a booking already sent is never sent again, so nothing double-books).
- **Effective-dated** — a value (a cap, a rate) carries the dates it applies from/to, so the system can use the figure that was in force on any given date.
- **Surrogate key /** `cuid` — a system-generated unique id for a record (not meaningful to a person).

- **Cascade-delete** — deleting a parent record automatically removes its child records (e.g. deleting a request removes its legs and estimates).
- **Soft reference** — a record points to another by a business code (e.g. a charging code) rather than a hard database link; resolved when read.
- **Roll-up** — combining the parts into a total (e.g. the per-line funding splits summed to the request's overall allocation).
- **Snapshot / version** — a frozen copy of the request as it was submitted/approved, kept unchanged for audit.
- **Fan-out** — a group trip produces one booking instruction per traveller.
- **Read model** — the pre-shaped data a screen reads (e.g. the dashboard list).
- **Canonical payload** — the standard message format exchanged with an external system.
- **Hard stop / Exception / Warning** — see §6.4: a blocker, an allowed-with-approval item, and an advisory note respectively.

Appendix B — Enumerations & constants

Request status, booking status, claim status, policy outcome, approver role, booking method/arrangement, expense category, TE treatment, ECS role — **full value lists in §4.4 and §4.3**. Band bounds `(lower, upper]` with first lower bound 0 inclusive. Tolerances, caps, ODA rates + tiers, workflow bands, policy thresholds — §5.2.

Appendix C — Source map & traceability

Where each behaviour lives in the source:

- **Routing** — `route.ts`
- **Policy** — `policy.ts` + `policyRules.ts` / `policyThresholds.ts`
- **Cost** — `pricing.ts`
- **Settlement** — `settlement.ts`
- **Duplicates** — `duplicates.ts`
- **TMC** — `tmcPayload.ts` / `tmcRetrieval.ts` / `integrations/tmc/*`
- **Guards** — `guards.ts`
- **Session** — `shared/session.ts`
- **ECS seam** — `shared/ecs/services.ts`
- **Settings** — `config/decisions.ts`
- **Rates** — `data/policyRates.ts`
- **Actions** — `modules/pretrip/actions.ts` + `modules/te/*`
- **Seed** — `modules/pretrip/demo.ts`

Design decisions trace to the companion Build Spec (DD-01…DD-30).

Appendix D — Decision register (design decisions)

Binding design decisions carried from the companion Build Spec:

- **DD-01** — config-driven routing
- **DD-02** — DOA always / traveller-selected
- **DD-03** — RO configurable
- **DD-04 / 05** — class entitlement; group per-traveller
- **DD-06** — accommodation never shared
- **DD-07** — per-line charging
- **DD-08** — claim auto-grant
- **DD-09** — one itinerary / trip-type preset
- **DD-10** — arrangement vs method
- **DD-11** — airport master = TMC source
- **DD-12** — no Reject (send back)
- **DD-13** — groups employees-only
- **DD-14** — one live authorisation (duplicate upfront)
- **DD-15** — failed-booking recovery
- **DD-16** — estimate defaults
- **DD-17** — 1 TR → N PNRs; TA-keyed inbound + PNR provenance
- **DD-18** — online vs offline hand-off
- **DD-19** — settlement channel; reimbursable = employee-settled
- **DD-20** — upgrade base / differential split (O-5)
- **DD-21** — guest PDPA via TMC
- **DD-22** — cancellation local-then-TMC
- **DD-23** — emergency lane
- **DD-24** — dashboard approvals + one-page summary
- **DD-25** — guest upgrade → Provost
- **DD-26** — automatic TMC hand-off
- **DD-27** — destination mandatory
- **DD-28** — separation of duties
- **DD-29** — policy-driven estimates (ODA & accommodation) auto-derived per city
- **DD-30** — CTA-settled charges: CTA→clearing master (posting-only lookup), claim reference lines auto-deducted, one-sided CTA charges reconciliation listing; guest bookings CTA-captured (no TE claim), same-or-separate guest CTA account

Open decision points: O-1 auto-grant tolerance figures · O-5 upgrade split percentages · O-R1 retention periods.

Appendix E — Index of derived requirements

Each requirement is stated in full at the point it is derived; this is the consolidated index.

- **TR-PT-01** — full lifecycle, server-enforced
- **TR-PT-02** — single integration boundary
- **TR-PT-03** — no admin approval bypass
- **TR-PT-04** — exact-decimal money
- **TR-PT-05** — server-side evaluation
- **TR-PT-06** — effective-dated FX, re-locked at submission
- **TR-PT-07** — audit + version snapshots
- **TR-PT-08** — idempotent hand-off / inbound
- **TR-PT-09** — immutable versions
- **TR-PT-10** — append-only audit
- **TR-PT-11** — as-approved from snapshot
- **TR-PT-12** — config-only policy change
- **TR-PT-13** — per-city auto-derived & re-derived estimates
- **TR-PT-14** — hard-stop / exception / warning semantics
- **TR-PT-15** — DOA always, not auto-selected
- **TR-PT-16** — no self-approval; no approver-origination
- **TR-PT-17** — reimbursable iff employee-fronted
- **TR-PT-18** — duplicate block + overlap warn, upfront
- **TR-PT-19** — TA-keyed retrieval + captured actuals
- **TR-PT-20** — emergency timing-only + named authoriser
- **TR-PT-21** — amendment re-runs route + re-issues TA
- **TR-PT-22** — production identity from ECS
- **TR-PT-23** — server-side state + remit re-check
- **TR-PT-24** — TMC interface (field-filtered, TA retrieval, fan-out)
- **TR-PT-25** — guest PDPA via TMC
- **TR-PT-26** — idempotent + audited hand-offs
- **TR-PT-27** — automated acceptance suite
- **TR-PT-28** — CTA account feed-reported; clearing resolved from a configurable CTA master (DD-30)

Appendix F — OutSystems sizing estimate

A planning estimate of the build's size in OutSystems terms, derived from the object inventory in this document. It is a **planning figure, not a formal Application Object (AO) audit** — the authoritative count is produced in Service Studio against the real factory and depends on the University's reuse

decisions. Two things are estimated: the **AO licensing count** (narrow — entities + integration methods + extensions) and the **development-object inventory** (broad — the basis for effort).

F.1 What counts as an Application Object

An AO is counted (1 each) for every **Entity** (database + static), every **REST API method** (exposed and consumed), every **SOAP method** (exposed and consumed), and every **Extension** action and entity. **Screens, Web Blocks, Server/Client Actions, Structures, Roles, Timers and Processes are NOT AOs** — they drive effort (§F.3), not the licensing count.

F.2 AO licensing estimate

The dominant variable is **reuse of existing ECS core**: this module *extends* ECS, so identity, DOA lines, charging accounts, GL/GST, FX and workflow bands SHOULD be consumed from the existing ECS core modules and add **0 AOs** to this application.

AO CATEGORY	ITEMS	REUSE-HEAVY	STANDALONE
Transactional entities	The 24 in §4.3 (TravelRequest, …Traveller, …Version, ItineraryLeg, EstimatedExpense, Accommodation/ODA Estimate, ChargingAllocation, PolicyCheck, ApprovalStep, TravelBooking, BookingSegment, BookingDeviation, Claim, ClaimApprovalStep, TEExpenseLine, TravelExpenseLink, IntegrationMessage, AuditEvent, Attachment, Provost, ModuleSettings, IntegrationField/GuardSetting)	**24**	**24**
New reference / static entities	Hotel caps (city + country), ODA rates, travel-class register, travel classes, travel purposes, locations (country/city/airport), workflow bands, tolerances/thresholds, high-risk destinations, TMC providers, rejection reasons — minus any already in ECS/TE core	~8	~14
Integration methods (consumed/exposed)	TMC hand-off · TMC TA-keyed inbound · TMC status · TMC cancel · SAP airfare posting · SAP GL allocation · visa-letter batch · notifications · (HR/Workday · DOA · FX if not reused)	~8	~16
Extension objects	SFTP/file (TMC workbook), PDF (approval pack)	~2	~4
Total AOs		**≈ 42**	**≈ 58**

Headline: ≈ 40–65 AOs, most likely 45–55, assuming ECS core exposes identity/DOA/charging/finance/FX as reusable modules. The 24 transactional entities are firm; reference entities and integration methods are the swing factors. If the TMC/OBT interface is **file-based** (SFTP workbook) rather than REST, part of that count shifts from integration methods into an Extension + Timer.

F.3 Development-object inventory (effort basis — not AO licensing)

OBJECT TYPE	ESTIMATE	BASIS
Entities	~34	24 transactional + ~8–14 reference/static
Reactive Web Screens	~23	The §11.2 route catalogue (6 wizard + queue/admin/finance/claim)
Web Blocks (reusable)	~18–25	Stepper, duplicate banner, itinerary builder, charging splitter, cost/breakdown table, route preview, PNR display, add-estimate panels, approval summary
Server Actions	~90–130	~49 action endpoints (§10.3) + decomposed domain logic (policy engine, routing, pricing + ODA tiers, settlement, duplicates, class entitlement, charging roll-up, auto-estimate derivation, claim workflow + auto-grant, TMC assemble/parse)
Client Actions	~50–80	Screen interactivity + live validation
Structures	~25–35	Canonical TMC payloads, DTOs, read models
Roles	9	§2.3
Timers / BPT processes	~4–6	Visa batch, TA expiry, booking deadline, TMC reconcile
Consumed REST/SOAP	~8–16	As §F.2

This is a **medium-to-large module**; the weight is logic density (policy engine, configurable routing, settlement, TMC contract, claim auto-grant), not screen count.

F.4 Drivers & assumptions

- **Reuse is the biggest driver** — consuming ECS core modules lands AOs ≈ 42; re-implementing masters/integrations pushes ≈ 58+.
- **TMC transport** — REST methods vs a file/SFTP workbook changes where the integration AOs land.
- **Reference data already in ECS/TE** (caps, ODA, class register) removes those AOs.
- Effort SHOULD be derived from §F.3 using the vendor's own OutSystems delivery velocity; this document does not assert person-days.

O-S1: confirm which ECS core modules (identity, DOA, charging, finance, FX, workflow bands, locations) are available for reuse — this sets the AO count.

F.5 Line-itemised AO register

The full per-object list behind §F.2. Default flags are the **reuse-heavy preset** (reuse every candidate); the interactive worksheet (companion artifact) lets each line be re-flagged. **Default billable AOs: 38** · all-new: 61 objects.

Transactional entities (owned by this module)

#	OBJECT	AO TYPE	BASIS	REUSE	AO
1	TravelRequest	Entity	Header & lifecycle (NEW)	—	1
2	TravelRequestTraveller	Entity	Group members (NEW)	—	1
3	TravelRequestVersion	Entity	Immutable snapshots (NEW)	—	1
4	ItineraryLeg	Entity	Multi-leg itinerary (NEW)	—	1
5	EstimatedExpense	Entity	Cost estimate lines (NEW)	—	1
6	AccommodationEstimate	Entity	Hotel + cap result (NEW)	—	1
7	ODAEstimate	Entity	ODA by city/dates (NEW)	—	1
8	ChargingAllocation	Entity	Funding split (NEW)	—	1
9	PolicyCheck	Entity	Policy engine output (NEW)	—	1
10	ApprovalStep	Entity	Approval route (NEW)	—	1
11	TravelBooking	Entity	Booking header (PNR) (NEW)	—	1
12	BookingSegment	Entity	Air/hotel segments (NEW)	—	1
13	BookingDeviation	Entity	Approval-to-booking variance (NEW)	—	1
14	TravelExpenseClaim	Entity	Seeded TE claim (NEW)	—	1
15	ClaimApprovalStep	Entity	Claim route (NEW)	—	1
16	TEExpenseLine	Entity	Claim lines (NEW)	—	1
17	TravelExpenseLink	Entity	TR↔claim link (NEW)	—	1
18	IntegrationMessage	Entity	TMC payload log (NEW)	—	1
19	AuditEvent	Entity	Append-only audit (NEW)	—	1
20	Attachment	Entity	Evidence metadata (NEW)	—	1
21	Provost	Entity	Guest-upgrade authority (NEW)	—	1
22	ModuleSettings	Entity	Runtime config (NEW)	—	1
23	IntegrationFieldSetting	Entity	Contract field override (NEW)	—	1
24	IntegrationGuardSetting	Entity	Contract guard override (NEW)	—	1

Subtotal — Transactional entities (owned by this module): 24 billable AO of 24 objects (0 reused).

Reference / static entities (reuse candidates)

#	OBJECT	AO TYPE	BASIS	REUSE	AO
25	Employee	Entity	HR identity / appointments (REUSE)	reuse	0
26	Department	Entity	Org + DOA line (REUSE)	reuse	0
27	ChargingCode	Entity	CC/WBS master (REUSE)	reuse	0
28	ExpenseType	Entity	TE expense-type master (REUSE)	reuse	0
29	GstCode / GLAccount	Entity	Finance master (REUSE)	reuse	0
30	FxRate	Entity	FX master (REUSE)	reuse	0
31	WorkflowBand	Entity	Amount→DOA tier bands (REUSE)	reuse	0
32	TravelClass	Entity	Class master (REUSE)	reuse	0
33	Delegation	Entity	Delegation master (REUSE)	reuse	0
34	RejectionReason	Entity	Send-back reasons (REUSE)	reuse	0
35	Country	Entity	Location master (REUSE)	reuse	0
36	City	Entity	Location master (REUSE)	reuse	0
37	Airport	Entity	IATA master (GDS sync) (REUSE)	reuse	0
38	TravelPurpose	Entity	Travel-specific (NEW)	—	1
39	HotelCap (city)	Entity	Travel policy cap (NEW)	—	1
40	HotelCap (country)	Entity	Travel policy cap (NEW)	—	1
41	OdaRate	Entity	ODA daily + monthly (NEW)	—	1
42	TravelClassRegister	Entity	Class entitlement register (NEW)	—	1
43	HighRiskDestination	Entity	Risk advisory master (NEW)	—	1
44	TmcProvider	Entity	TMC provider master (NEW)	—	1

Subtotal — Reference / static entities (reuse candidates): 7 billable AO of 20 objects (13 reused).

Integration methods (REST/SOAP)

#	OBJECT	AO TYPE	BASIS	REUSE	AO
45	TMC — Send booking instruction	REST	Outbound hand-off (NEW)	—	1
46	TMC — Retrieve bookings by TA	REST	TA-keyed inbound (all PNRs) (NEW)	—	1
47	TMC — Booking status callback	REST	Status webhook (NEW)	—	1
48	TMC — Cancellation request	REST	Cancel (two-way) (NEW)	—	1
49	SAP — Airfare direct posting	REST	DIRECT_SAP posting (NEW)	—	1
50	SAP — Allocation → GL posting	REST	Finance posting (REUSE)	reuse	0
51	SAP — Charging account read	REST	CC/WBS read (REUSE)	reuse	0
52	HR/Workday — Employee & appointment	REST	Identity read (REUSE)	reuse	0
53	HR/Workday — Reporting line	REST	RO read (REUSE)	reuse	0
54	HR/Workday — Home-dept cost object	REST	Auto-allocate read (REUSE)	reuse	0
55	DOA — Line + amount band	REST	DOA candidates read (REUSE)	reuse	0
56	FX — Rate read	REST	Effective-dated FX (REUSE)	reuse	0
57	Immigration — Visa-letter batch	REST	Outbound daily batch (NEW)	—	1
58	Notifications — Send	REST	Approval/booking alerts (REUSE)	reuse	0
59	Location/GDS — Airport sync	REST	Airport feed sync (REUSE)	reuse	0

Subtotal — Integration methods (REST/SOAP): 6 billable AO of 15 objects (9 reused).

Extensions

#	OBJECT	AO TYPE	BASIS	REUSE	AO
60	SFTP / file adapter (TMC workbook)	Extension	File transport (if file-based) (NEW)	—	1
61	PDF generator (approval pack)	Extension	Forge component candidate (REUSE)	reuse	0

Subtotal — Extensions: 1 billable AO of 2 objects (1 reused).

Register total: 38 billable AOs · 61 objects (23 reused) — reuse-heavy default. Standalone (all reuse off) = 61 AOs. The headline planning range remains ~40–65 AOs.

This specification describes the intended build. Final design may change at a formal discovery / detailed-design phase with the implementation vendor; material changes are tracked by version.
